Architectural Guardrails: An Opinionated Framework for Preventing Structural Decay in Greenfield Research Software

Art Ó Liatháin
22363092@studentmail.ul.ie
University of Limerick

May 2026

Keywords Static Analysis · Machine Learning · Architectural Technical Debt · Maintability · Research Software · Temporal Coupling · Random Forests · Heuristic Based Dataset Generation

Supervisors

Primary Supervisor
Eoin O'Brien
Department of Computer Science and Information Systems
University of Limerick

Secondary Supervisor
Colin Venters
Department of Computer Science and Information Systems
University of Limerick

1 Abstract

There is a fundamental misalignment between academic funding cycles and software lifecycles, resulting in "disposable" research code that is neither reproducible nor maintainable, creating an ecosystem in which research projects lay abandoned after being used, unable to be extended or reused requiring reimplementation for it to be used for further research. Researchers do not have the time or capabilities to learn programming to the degree needed to write truly maintainable code while staying within funding cycles. While static analysis is often proposed as a solution to raise the "code quality floor," traditional universal expert derived rules frequently fail to account for the unique evolutionary patterns of scientific software.

This study employs a bespoke methodology to create domain specific maintainability datasets through historical repository metadata, leveraging this dataset of over 200 research repositories and 9,000 C++ artifacts to train interpretable machine learning models. By empirically calculating Hub Score (A composite metric based on temporal metadata), the approach shifts the paradigm of code governance from subjective expert derived rules toward data-driven thresholds rooted in the actual evolutionary history of scientific software. Validating this by benchmarking derived rules against industry-standard tools using CodeScene on CERN research repositories.

The findings reveal that while machine learning models (Decision Trees and Random Forests) trained on file-level metrics achieved high accuracy, they yielded low F1 scores and produced uninterpretable rules too complex for manual developer application. However, external validation against CERN repositories showed a significant shift in Cohen's Kappa from "Fair" (0.224) to "Moderate" (0.414) agreement with industry-standard benchmarks (CodeScene). This improvement proves that a measurable structural temporal correlation exists which machine learning can bridge, even when using labels derived solely from version control metadata, validating the use of these temporal metrics for further training of machine learning methods.

Based on the findings this study suggest that the future of research software maintainability lies not in human-centric linting where interpretable rules limit model potential, but in AI focused automation where black-box models identify and resolve high-risk architectural hotspots within the development loop before a researcher can even see them. This works to achieve the goal that scientific artifacts remain in use after the initial grant period concludes allowing researchers to work towards the future rather than reimplementing the past.

Declarations

1.1 Declaration of Originality

I herewith declare that I have produced this paper without the prohibited assistance of third parties and without making use of aids other than those specified; notions taken over directly or indirectly from other sources have been identified as such. This paper has not previously been presented in identical or similar form to any other Irish or foreign examination board.

The thesis work was produced under the supervision of Eoin O'Brien and Colin Venters at University of Limerick.

Art Ó Liatháin
Limerick, 2026

1.2 AI Declaration

I herewith declare that I have used artificial intelligence to produce my project and/or report in the following ways: I further declare that I have discussed this use of artificial intelligence with my supervisor and received permission to use it.

Art Ó Liatháin
Limerick, 2026

1.3 Ethics Declaration

I herewith declare that my project does not involve human participants in any way and that I therefore was not required to submit an ethics application.

Art Ó Liatháin
Limerick, 2026

Acknowledgements

I would like to express my sincere gratitude to my primary supervisor, Eoin, and my secondary supervisor, Colin, for their guidance, patience, and insight throughout the development of this research. Making sure I stayed on the right track, not jumping down into any rabbit holes and always stepped in to help me whenever I needed it.

Special thanks go to my partner, family and friends for their unwavering support and encouragement during the entirety of this project, stopping me from procrastinating and their belief in me made sure this study achieved its full potential.

Finally, I would like to acknowledge the open-source contributors of the repositories analyzed in this study, without whom this data-driven research would not have been possible (*Journal of Open Source Software*, no date; *Home | CERN*, no date).

Contents

1 Abstract . . . 3
1.1 Declaration of Originality . . . 4
1.2 AI Declaration . . . 4
1.3 Ethics Declaration . . . 4
2 Introduction . . . 1
3 Background Research . . . 3
3.1 The Current State of Research Software . . . 3
3.1.1 The Importance of Guidelines . . . 3
3.1.2 Funding and Research Software . . . 3
3.1.3 The need for Maintainability . . . 4
3.2 Maintainability . . . 4
3.2.1 What does it mean to be Maintainable? . . . 4
3.2.2 Measuring Maintainability . . . 5
3.2.3 Code Churn's impact on Maintainability . . . 7
3.3 Static Analysis . . . 8
3.3.1 Current Methods . . . 8
3.3.2 Issues with Modern Approaches . . . 8
3.3.3 Rules as Data Approaches . . . 9
3.4 Machine Learning . . . 10
3.4.1 Decision Trees . . . 10
3.4.2 Random Forest . . . 10
3.4.3 Applications to Maintainability Detection . . . 11
3.5 Summary . . . 12
4 Methodology . . . 14
4.1 Dataset Selection and Acquisition . . . 14
4.1.1 Data collectiowere . . . 14
4.1.2 Dataset Co Change Graph Creation . . . 16
4.2 Dataset Labeling . . . 17
4.2.1 Hub Score . . . 18
4.2.2 File classification and Oversampling . . . 20
4.2.3 Dataset Verification Method . . . 21
4.2.4 Addressing Data Leakage . . . 22
4.2.5 Dataset Summary . . . 22
4.3 Feature Engineering . . . 23
4.4 Model Implementation and Training . . . 25
4.5 Rule Derivation Process . . . 25
4.6 Evaluation Framework . . . 26
4.7 Threats to Validity . . . 26
4.7.1 Construct Validity . . . 26
4.7.2 Internal Validity . . . 26
4.7.3 External Validity . . . 27

4.8 Summary ... 27
5 Results ... 28
5.1 Hub Score Comparison with CodeScene ... 28
5.2 Machine Learning Models ... 28
5.2.1 Model Configuration ... 28
5.2.2 Model Performance ... 29
5.2.3 Feature Importance ... 30
5.2.4 Extracted rules ... 31
5.2.5 External Validation ... 33
5.3 Summary of Findings ... 34
5.3.1 Autonomous Labeling Accuracy (RQ1) ... 34
5.3.2 Opinionated Interpretable Guardrail Rule Derivation (RQ2) ... 34
6 Discussion ... 36
6.1 Evolutionary Risk and Structural Metrics(RQ1) ... 36
6.1.1 Temporal Metrics as a Proxy for Structural Decay ... 36
6.1.2 Implications for Large-Scale Research ... 36
6.2 The Structural-Temporal Mismatch (RQ2) ... 36
6.2.1 Confidence in Actionable Guardrails ... 37
6.3 What this means for Research Software (RQ2) ... 38
6.3.1 The Empirical Ground Truth ... 38
6.3.2 The Changing Landscape from Humans to AI ... 38
6.3.3 Analytical Black Boxes ... 38
7 Conclusion ... 40
7.1 Contributions ... 41
7.1.1 Primary Contributions ... 41
7.1.2 Secondary Contributions (Technical Artifacts) ... 41
7.2 Future Work ... 42
Appendix ... 43
7.3 Software and Data Availability ... 43
Bibliography ... 44

List of Figures

Figure 1 Pre filter Distribution of Collaborators ... 15
Figure 2 Pre filter Distribution of Commits ... 16
Figure 3 Hub Score Distribution across metrics: **a)** Commit Count, **b)** Partner Count, **c)** Code Churn, and **d)** Average Coupling. ... 19
Figure 4 Hub Score Distribution: a)Hub Score distribution across all .cpp samples , b) Distribution of high and low risk files ... 21
Figure 5 High risk distribution across all repositories ... 23
Figure 6 Confusion Matrices of predicted metrics: **a)** Decision Tree SMOTE, **b)** Random Forest SMOTE, ... 30

Figure 7 Confusion Matrices of predicted metrics: **a)** Decision Tree Feature Importance , **b)** Random Forest Feature Importance, ... 31
Figure 8 Decision Tree Visual representation 32

List of Tables

Table 1 Dataset file counts at each step 21
Table 2 Extracted code metrics (Ardito *et al.*, 2020) 24
Table 3 High risk identification comparison Hub Score and CodeScene 28
Table 4 Model hyperparameters 29
Table 5 Model Performance Comparison 29
Table 6 High risk identification comparison SMOTE Random Forest and CodeScene . 33

2 Introduction

Software is no longer a tool that lives on the edge of research. Research software has become the primary infrastructure upon which scientific claims are built, helping progress discoveries ranging from the intricate algorithms of genome sequencing to the petabyte-scale data pipelines of the Large Hadron Collider. Software has grown to such an importance in research, its significance cannot be ignored, yet why is software overlooked time and time again?

Buggy software produces incorrect results, incorrect results lead to incorrect conclusions which in turn misinforms future research. This is a pervasive issue in research publishing, as the "publish or perish" model in the current landscape encourages researchers to hack together a solution to get the results as soon as possible, naturally leading to more bugs being created. A core issue is what comes after, verification is difficult, 90% of researchers relied on learning code through self teaching (Hannay *et al.*, 2009), meaning fixing, much less identifying bugs, is something outside of the technical scope for many researchers and peer reviewers. Worse still, the social pressure to maintain a "perfect" record can lead to a culture of silence, where identified flaws are buried to avoid the perceived stigma of retraction.

To combat this issue reproducibility should be a cornerstone for allowing research to be verified and trusted. The current state of research software is anything but, (Trisovic *et al.*, 2022) identified that across 9,000 R files only 25% of them could run without error or the need for modification. A key note here is there was no comparison to the original research to ensure results matched only to see if the files could execute without error. This is a shocking result, for the reproducibility to be so low it has long term cascading effects: if code lacks longevity future researchers will need to re implement previous work to be able to start the next step, slowing future research. It also raises an insidious thought:

If researchers produce results using a code artifact that cannot be reproduced, the results they produced may never have existed in the first place.

The research community as a whole has made an effort towards reproducibility in research software (Jiménez *et al.*, 2017; Wilson *et al.*, 2017). The issue is reproducibility is reactive, only concerned if we can run the code now but this study posits that this is a symptom of a deeper unaddressed requirement, Maintainability. Maintainability is proactive ensuring the code not only lives now but ensures the code can be understood, modified and utilised well and efficiently for future research.

Tools like static code analysis are imperative to research software as a whole to raise code quality. Allowing automatic analysis to be conducted on large scale repositories catching, issues early before they grow to be a larger problem. These tools not only allow for code quality to be understood but they facilitate a targeted approach to refactoring. Unfortunately, in the application to the research domain there is a key issue: Warnings generated by these tools are often ignored due to the high volume generated (Johnson *et*

al., 2013), leading to a situation where without a knowledge of programming to identify fact from fiction potential refactoring gets ignored.

Warnings are not enough; there needs to be an opinionated system that uses empirical data to accurately predict if a file will become a maintenance issue before it is added to a larger codebase. Expert-driven rules attempt to generalise across all domains, but each domain has unique patterns and approaches which need to be targeted differently. A tool used for particle movement tracking requires a fundamentally different set of quality metrics than a standard enterprise web application. A general approach ignores the nuance of each domain, a focused set of rules for each domain would reduce the number of false positives, increasing the standard of code within each domain.

This study seeks to answer two questions:

RQ1: To what extent can historical version control metadata be used to autonomously label maintainability risk in domain-specific repositories?

RQ2: To what extent can accurate, opinionated, and interpretable rules be derived from structural code metrics to predict these historically identified risks using machine learning?

To tackle this, an explainable machine learning (ML) approach using Decision Trees (DT) and Random Forests(RF) will be used. The primary objective is to leverage these models to derive a foundational set of domain specific static analysis rules tailored to the structural nuances of research software.

A limitation that has been present previously for machine learning for static analysis is requiring a manually labeled dataset. Datasets such as PROMISE (Boetticher, Menzies and Ostrand, 2007) require a commitment from many developers to accurately label a small sample set, meaning that a domain specific approach would be time consuming and effort intensive. To combat this, an automatic approach using heuristic-based labelling methods will be utilised to quickly curate a high-quantity, domain specific dataset from research repositories sampling from over 200+ different research repositories resulting in a high quality dataset of over 9,000 files. Allowing the ML models to learn patterns specific to research software, increasing the confidence in the resulting quality gate.

Ultimately, this study moves beyond industry standard domain agnostic static analysis, offering instead a framework of opinionated static analysis guardrails. Guardrails that are empirically derived, domain specific, and built to protect the long term usability and verifiability of research software. By validating these rules against industry standard benchmarks,specifically CodeScene, using unseen repositories from the CERN software ecosystem, this study demonstrates a scalable path toward proactive software maintainability in research.

3 Background Research

3.1 The Current State of Research Software

Research software is software made to investigate new ideas and concepts. This comes with one very prominent issue, researchers who are leading the way to new concepts are generally not programmers by trade, 90% of researchers and graduates rely on self teaching in programming (Hannay *et al.*, 2009). Software is made without maintainability in mind only following the check of "It works on my computer". This is best observed by (Trisovic *et al.*, 2022) where over 9000 R files were analysed from the Dataverse Project (http://dataverse.org/). In this study only 25% of R projects were runnable initially, after performing code cleaning, inferring dependencies and altering hardcoded paths 56% were able to be run. This is a shocking finding, as this is irrespective of the code output, only having a successful run of the code was incredibly inconsistent. (Trisovic *et al.*, 2022) did note that journals are beginning to mandate better quality standards for code through research software standards to particularly focusing on reproducibility to ensure the results published by the papers are verifiable. Yet the guidelines for what is needed in terms of quality metrics for the code is still unclear.

3.1.1 The Importance of Guidelines

Researchers and graduates need clear guidance on how to create and maintain a large software project due to their lack of formal training (Hannay *et al.*, 2009). Unfortunately, mainstream research software guidance remains overwhelmingly focused on reproducibility (Wilson, 2006; Wilson *et al.*, 2017; Jiménez *et al.*, 2017; Marwick, 2017), often relegating other critical facets (such as maintainability and performance) to secondary considerations. While industry utilizes mature tooling and standards to balance reproducibility with code health, academic guidance often fails to provide actionable criteria for software architecture or long term maintenance. This lack of structural guidance directly contradicts the sustainability goals for research software proposed by (Howison and Herbsleb, 2014).

3.1.2 Funding and Research Software

A primary driver of this structural decay is the fundamental mismatch between the nature of software development and the mechanisms of academic funding. Most research grants do not explicitly provision for dedicated Research Software Engineers (RSEs), treating code as a secondary byproduct rather than a core asset requiring professional attention (Goble, 2014).

Compounding this issue is that research software is made to fit a funding cycle without a care for the longevity of the codebase. Funding is fundamentally not suited for research software (Howison and Herbsleb, 2014) notes that unlike commercial products where revenue grows linearly with users, research grants are fixed and do not require or reward researchers for user acquisition. Funding pushes research software to focus on itself with no incentives outside of a research paper output. A flawed approach which (Howison and

Herbsleb, 2014) does address with two alternative solutions; Commercial models in which the research software itself transitions to a self sufficient project which keeps itself alive or peer-production models which is a similar approach to open-source. Both of these models have merit, they allow research software to live past the funding cycle and continue to improve over time.

3.1.3 The need for Maintainability

However, there is one key issue: both rely on the existence of a maintainable codebase. If a project is built with significant "technical debt" during the funding cycle, the cost of maintenance becomes an insurmountable barrier. A commercial spin-off will fail under the weight of high development costs, and a peer-production community will fail to form because outside contributors cannot navigate or extend a poorly structured codebase. Thus, without a foundation of coding standards, these sustainability models remain aspirational rather than achievable.

Addressing the core issue of research software longevity while adhering to the constraints governing research software as a whole is the gap this paper is seeking to fill. To create an opinionated static analysis tool that will ensure code written has a floor of quality it cannot dip below through objective quality gates. By automating the "quality check" process, the quality of research software maintainability can be raised without requiring researchers to become professional developers. Static analysis provides the immediate, clear guidance that current approaches lack, ensuring that software is not just reproducible today, but maintainable and extensible for the researchers of tomorrow.

3.2 Maintainability

To raise code quality, maintainability must be defined. Maintainability is a term used frequently in software engineering, there is no agreed upon definition on what maintainability is but ISO25010 defines it as "The degree of effectiveness and efficiency with which a product or system can be modified to improve it, correct it or adapt it to changes in environment, and in requirements". It defines that the sub-sections of maintainability are *Modularity, Re-usability, Analysability, Modifiability, Testability.* (*ISO 25010*, no date) This is a very broad definition which essentially defines requirements for how easily the system can be modified for change.

The lack of clear testable outcomes for each quality in the ISO standard leads to a conceptual overlap where maintainability becomes a "catch-all" category for non-functional requirements, making assessing what is truly maintainable very difficult. Meaning that while the standards are set out, they are subjective and can change from person to person, organisation to organisation defeating the purpose of standards in the first place. This is why clearly defining what maintainable code looks like is paramount to this paper.

3.2.1 What does it mean to be Maintainable?

To define code as maintainable, modularity is one key factor in the assessment. Modularity done correctly facilitates changeability through logical decomposition of functionality,

it may seem simple yet if done incorrectly can have long lasting consequences on the maintainability and longevity of a codebase.

D.L Parnas presented a study revealing how easily developers could fall into the pitfall of designing the system in a human like manner and the issues that arise from that approach (Parnas, 1972). Designing systems based on a humans natural flow through a system, converting those actions into a flowchart of logical operations is a method that creates code that is not resilient. While it may be modular on paper as every "step" would be logically separate the code can still be highly coupled through cross cutting dependencies which do not conform to the human flow, impacting the maintainability of the codebase.

An example of this would be for an ordering system, the flow would be:

Receive order -> Validate order -> Charge card -> Reserve stock -> Ship order -> Send receipt.

A flowchart style design might put all database writes in the "ship order" step because that's when a human thinks the order is complete. But if payment fails, refunds, or partial shipments happen later, every earlier step may need to be revised. In a more robust design, payment, inventory, and fulfillment each own their own data and expose stable interfaces, so changing one rule does not force a rewrite of the whole sequence.

The effect has been substantiated by (Cai *et al.*, 2025) with a direct correlation being made between coupling and maintainability overhead. The core purpose of modularisation should not be encapsulating subroutines into modules. Design decisions that are likely to change should serve as the foundational points to creating modular code, allowing subroutines to be compositions of modules. Taking modularity into account will serve as a solid baseline to assess the maintainability of a codebase and files.

3.2.2 Measuring Maintainability

As maintainability is an abstract concept, various frameworks have attempted to reduce it to a concrete numerical metric. An early approach in this area is the Maintainability Index (MI), which calculates a score based on a weighted combination of Halstead Volume, cyclomatic complexity, lines of code (LOC), and comment density (Oman and Hagemeister, 1992). The formula typically utilized for this assessment is:

$$\mathrm{MI} = 171 - 5.2\ln(V) - 0.23(G) - 16.2\ln(\mathrm{LOC}) + 50\sin\left(\sqrt{2.46 \times C}\right) \tag{1}$$

Where:

- V represents **Halstead Volume**;
- G is **Cyclomatic Complexity**;
- C is the **percentage of comment lines**.

(Oman and Hagemeister, 1992) validated this metric through feedback from 16 developers and empirical testing on industrial systems, including those at Hewlett-Packard and various defense contractors. This methodology provided a pragmatic tool for engineering managers to prioritize maintenance efforts by assigning a tangible value to code quality. While the

MI provides a high-level overview of code density, it suffers from what can be described as "semantic blindness". Metrics such as Cyclomatic Complexity and Halstead Volume analyze the control flow and token count of a file but fail to interpret the structural intent or the relationships between components. For the purposes of this paper this lack of overarching analysis will be referred to as "blind metrics". Utilising blind metrics allows for efficient and quick analysis but it opens the door to gaming the system in a sense, where developers could reach extremely maintainable scores syntactically while semantically being unmaintainable, thereby abusing the formula.

For example, A script can maintain a high MI score due to low complexity and short length, yet contain architectural flaws such as "God Objects"(A singular class or file in which does the bulk of the business logic generally being thousands of lines long) or tight coupling to external datasets that inhibit reuse. The formula was a product of its time: finely tuned to the teams and projects it was applied to, in the modern landscape the current formula would definitely not be accurate with how modern languages have evolved. Even though the formula could be adapted to a modern context it is clear that even a modernised version would suffer from blind metrics and is not something that should be used as only counting lines cannot yield results on the true quality of code.

An approach that would apply in a more modern context was proposed by (Xiao *et al.*, 2016), of identifying architectural debt through evolutionary analysis of a codebase (Xiao *et al.*, 2016). Where architectural debt can be used as a measure of maintainability it quantifies debt through a formalized Debt Model, using regression analysis to capture the growth rate of maintenance costs over time. The paper proposed that there are four key architectural patterns that are the main proponents of ATD, Hub, Anchor Submissive, Anchor Dominant, and Modularity Violation. These patterns are all based on combining structural context alongside evolutionary context to create a holistic view of the codebase.

Hub: Characterised by strong, mutual coupling where the anchor file and its members have structural dependencies in both directions and strong commit coupling in at least one. This pattern often represents "spaghetti code" where a central file is overloaded with responsibilities.

Anchor Submissive: Occurs when members structurally depend on an anchor, but the anchor is historically submissive, changing whenever the members change.This typically indicates an unstable interface being forced to change by its clients.

Anchor Dominant: The reverse of submissive, where members structurally depend on the anchor, and the anchor historically dominates them, frequently propagating changes outward to its dependents.

Modularity Violation: Identified as the most common and expensive form of architectural debt across various projects. It is unique because it involves files that have no explicit structural dependencies but are frequently changed together in the project's commit history.

The four patterns were measured by performing a longitudinal study on the commit histories of large open source codebases. Using tools such as Understand and Titan to derive the relevant pattern by calculating the commit coefficient between every file.

A metric which relates to the chance of a commit on one file will require a commit on another. For example, given files A, B, and C with commit histories of {A,B} and {A,C}:

- If B or C is modified: 100% chance that A will also be modified
- If A is modified: only 50% chance that B or C will be modified

This would then be called a fileset, this would be extrapolated over the entire commit history and codebase creating many filesets. This would allow the tool to extract semantic relations between files that are not apparent structurally. The study was able to compound this effect by measuring the number of commits labeled as bug fixes against the number of feature commits, allowing simple data analytics to measure the amount of maintenance debt that each fileset would have. Using this method, high maintenance filesets can be labelled and evaluated as a quantitative metric. The core value of their work is the identification of Modularity Violations: instances where files are forced to change in tandem despite having no logical reason to do so. In the "wild west" of research software, where commit messages are often uninformative, it is not possible to rely on a human to label a change as a "bug fix."

A substitute that this paper explores is replacing missing semantic data with a heuristic based approach where file risk gets identified through metrics. This necessitates shifting the focus from why a file changed to how much it changed and what it changed with. Setting the focus for the heuristics to be temporal while ignoring the structural. Using the scale of the data to supplement the natural loss of fidelity that this created, naturally leading to the study of Code Churn.

3.2.3 Code Churn's impact on Maintainability

Code churn is the rate of change of lines of code in a file. A file with high churn might have an incredibly high number of additions and deletions while remaining small in terms of lines of code. This is an ATD hotspot when assessing points of interest in terms of maintainability in codebases. (Faragó, Hegedűs and Ferenc, 2015) is a paper which assessed how code churn affects the maintainability of codebases.

The maintainability of the code was measured using ColumbusQM probabilistic software quality model (Bakota *et al.*, 2011). The paper was able to assess the levels of maintainability per file in proportion to the amount of code changes, drawing the conclusion that code that has high churn is code in which it is harder to maintain. This conclusion makes sense, it reaffirms that poorly architected code will need more changes which increases the maintainability burden of the code. The method in which the code churn was measured is quite useful and will be the approach taken for this study.

Unfortunately there is one key issue with this paper, the use of ColumbusQM for assessing the maintainability of the codebase. ColumbusQM is a statistical machine which takes

metrics such as lines of code, styling, coupling, number of incoming invocations etc.. Aggregates them which then computes a numerical output. The metrics such as styling are something so subjective and minor that the inclusion as one of the key metrics in the evaluation undermines the whole statistical model. Taking into account all of the evaluation metrics this is a case of blind assessment where structural relations or evolutionary dependencies are not measured and only a snapshot of the codebase is measured.

Taking these points into account, the key takeaway from (Faragó, Hegedűs and Ferenc, 2015) is the methodology regarding code churn, rather than its conclusions on maintainability. By identifying the intersection of Modularity Violations (Structural) and High Churn (Temporal), a composite metric can be created that avoids the "blindness" of ColumbusQM. This allows us to quantify architectural decay using only version control metadata, remaining entirely agnostic to both the quality of documentation and the subjective "style" of the researcher. Giving us the foundation of files to be used to create opinionated static analysis tooling.

3.3 Static Analysis

The transition from historical temporal analysis to active development requires a mechanism that can evaluate code in its current state, rather than its past. Static Analysis serves as this mechanism, offering a storied field of research focused on improving code quality by examining source code without execution. While the temporal analysis identifies the "symptoms" of architectural decay through commit history, static analysis allows us to identify specific structural patterns that lead to high maintenance overhead.

3.3.1 Current Methods

Modern methods for static analysis include data flow analysis, syntactic pattern matching, and abstract interpretation (Gosain and Sharma, 2015). These methods have evolved from simple token-level scanning to sophisticated project level reasoning, such as path analysis and control-flow graph mapping. Current state of the art tooling such as CodeScene has made strides in moving towards behavioural code analysis. CodeScene's creation of "refactoring targets", a system in which traditional static analysis is used to identify high risk files is overlaid with a heatmap of developer activity to focus in on problematic active files (Tornhill, 2024). Addressing a fundamental issue of traditional static analysis: the "urgency" gap. This approach is grounded in the principle that tech debt is only an immediate risk if it exists in active code. These "refactoring targets" provide a pragmatic prioritisation of maintenance effort allowing developers to focus on the 1-3% of the codebase that typically accounts for the majority of maintenance overhead (Tornhill, 2024).

3.3.2 Issues with Modern Approaches

Despite this increase in analytic capabilities, noise pollution in which there are too many warning messages discouraging any actions being taken to remedy them, the same issue which plagued Lint (Johnson and Hill, 1978) is still an issue to this day. Thereby polluting the developer experience creating friction and actively discouraging developers from using

the tools as it becomes more effort to sift through and find the real bugs than finding them normally (Johnson *et al.*, 2013). This points to the conclusion that the research community, especially due to the lack of experience in the field (Hannay *et al.*, 2009) would have more difficulties differentiating fact from fiction and discouraging them from using static analysis at all. This concretely highlights the difficulties of modern static analysis tooling in a research context. The focus on a general one size fits all tool means that it lacks the ability to be opinionated by default. The gap this presents is one this study will explore, where research software would have a bespoke set of curated rules reducing noise and guaranteeing improvements in code quality once executed.

This study posits that the key issue with the analysers in the case of false positives is not the rules but how they were derived. Historically most academic research in software engineering has required significant empirical evidence to be considered by industry (Weyuker, 2011), yet the current landscape is dominated by expert analysed rules. The challenge of software maintainability with static analysis is elusive, (Borg, Ezzouhri and Tornhill, 2024) highlights this, critiquing the "ghost echoes" produced by traditional expert-derived static analysis. These echoes are not objective truths, but rather technical artifacts, static rules that linger in codebases despite often failing to align with the lived experience and thoughts of human developers.

By benchmarking machine learning models against human judgment, the study reveals a fundamental difference: expert rules frequently draw "hard lines" that humans do not actually perceive as problematic. This suggests that codebase governance based on these traditional metrics is less an exercise in objective optimization and more an adherence to institutional haunting. The precedent set here is that for codebases to remain truly maintainable, developers must move beyond the rigid, subjective opinions of experts and toward empirically validated models that reflect the nuanced, non-deterministic reality of human-centric code health"(Borg, Ezzouhri and Tornhill, 2024).

3.3.3 Rules as Data Approaches

The limitations of expert derived thresholds are clear and reveal the need for a paradigm shift from deductive to inductive static analysis. Traditional tooling is deductive, applying a pre-defined expert rule as a universal threshold to a specific file to deduce a "bug" or "code smell". In contrast, a rules as data approach treats the repository itself as the source of truth with its history as well as the current state being imperative to identifying architectural technical debt. (Xiao *et al.*, 2016; Borg, Ezzouhri and Tornhill, 2024)

Machine learning has the capabilities to identify patterns in code, how it evolves, how it decays, with this holistic view an inductive approach can be taken. In which the "rules" are taken not from expert opinion but from data taken from the wider domain. Creating curated opinionated rules tailored to a specific domain, minimising the "ghost echoes" of irrelevant "universal" rules created for industry, replacing them with empirically validated opinionated guardrails on code quality.

3.4 Machine Learning

Machine learning is a pivotal technology within the context of this paper, allowing rigorous statistical analysis to be carried out to create objective metrics for maintainability rules. These algorithms operate by minimizing an objective function(A mathematical representation of error), an example would be decision trees, this is achieved through greedy splitting, where the algorithm recursively partitions data to minimize "impurity" (uncertainty) at each step. By optimizing for the lowest possible error, the model becomes increasingly proficient at identifying patterns within the problem domain. There are many approaches and models available but due to an emphasis put on human readability, Decision Tree and Random Forest models best fit this use case.

3.4.1 Decision Trees

Decision Trees are a standout choice when it comes to interpretable machine learning models. They consistently rank highly within classifier models while still supporting rule extraction which aids readability. Decision Trees represent classifiers as hierarchical IF-THEN rules, where each root-to-leaf path corresponds to a conjunction of feature thresholds that directly maps to executable static analysis checks.

A concrete example of this would be IF nesting > 3 THEN unmaintainable; in the case of nesting being over 3 then the code would be labeled unmaintainable. This simplistic example would be scaled up much larger with many more decision and leaf nodes creating a classifier based on simple decisions at each step.

Naturally there are some drawbacks to this approach, most prominently the overfitting problem. An overfitted model reflects the structure of the training data set too closely, even though a model appears to be accurate on training data, it may be much less accurate when applied to a current data set (Khoshgoftaar and Allen, 2001). This issue restricts their usage in current maintainability prediction methods, (Bluemke and Borsukiewicz, 2023) shows that Decision Trees can prove useful in a baseline analysis of maintainability predictors.

Despite limitations, Decision Tree rules provide the simple interpretability missing from Random Forests, enabling manual validation of learned static checks against domain knowledge. Pruning or ensemble averaging in random forests addresses overfitting to a degree while retaining path-derived rules for analysis (Quinlan, 1993).

3.4.2 Random Forest

Random Forest is an approach derived from Decision Trees and a popular classifier machine learning approach. Relying on creating a set of Decision Trees (called a forest) each of which vote on the outcome to choose a class for the classifier, it creates a method that leverages the law of large numbers to deal with the overfitting problem in Decision Trees (Breiman, 2001) in addition to increasing accuracy. The method in which overfitting can be largely ignored is that there are many trees voting, every one could potentially be overfitted yet the whole forest remains generalised to the problem as every tree would be overfitted to a different feature.

Random forests average better results theoretically (Breiman, 2001) and practically in the field of maintainability research (Bluemke and Borsukiewicz, 2023) over Decision Trees. The key drawbacks to Random Forests are both the increased computational costs and the black box view when it comes to readability.

Modern research (Haddouchi and Berrado, 2025) has made progress in creating methods of rule extraction for Random Forests as before the size of the forests rendered them as a black box approach. (Haddouchi and Berrado, 2025) focused on deriving a rule ensemble based on the calculated weighted importance of the trees, allowing for a dimensional reduction method to be applied to the forest. Within the study an average factor of 300x reduction to the number of rules per target class both in binary and multi classification problems was observed while retaining a 93% accuracy compared to the full forest's 95% accuracy. Allowing a highly optimised set of rules to be generated for both performance and explainability, leveraging this rule extraction method for Random Forests is pivotal to create a rule set that properly generalises across projects while retaining the core idea and performance of the model and keeping with the core ideal of readability.

3.4.3 Applications to Maintainability Detection

The application of machine learning to maintainability detection enables a data-driven approach to identifying code structures that hinder long term quality and evolution. Traditional static analysis methods rely on human curated heuristics, which often fail to generalise across diverse projects or languages. In contrast, machine learning can infer maintainability rules directly from empirical evidence, allowing patterns of poor or high maintainability to emerge statistically rather than being predefined.

State of the Art(Sota) models achieve impressive results in classification of low maintainability files. (Bertrand *et al.*, 2023) achieved an 82% F1 score with an AdaBoost classifier, while (Bluemke and Borsukiewicz, 2023) achieved 93% F1 score with a random forest classifier and (Schnappinger *et al.*, 2019) achieved an F1 score of 80% using a J48 classifier across three labels.

However, each approach shares a critical limitation, they rely on human experts to create a ground truth of annotated data. PROMISE (Boetticher, Menzies and Ostrand, 2007), MainData (Schnappinger, Fietzke and Pretschner, 2020) and a custom expert labelled dataset (Schnappinger *et al.*, 2019) were used in these studies. This introduces a certain degree of subjectivity which can be minimised by increasing the number of reviewers although limiting scale. These datasets are also by design due to the time commitment labelling, older and cannot reflect the current programming landscape. This limitation is the key motivation for an algorithmic approach to labelling, allowing us to create a dataset based on current codebases at scale.

A crucial step in applying these machine learning models to maintainability detection is the transformation of source code into a machine readable representation. Abstract Syntax Trees (ASTs) provide this structure, encoding the syntactic and hierarchical relationships between program elements. By traversing or analysing the AST, a wide range of numerical

and categorical features can be extracted: such as nesting depth, branching density, or average method size, which serve as the input features for machine learning models (Bertrand, Favier and Andre, 2022). Various learning models can then be trained on these AST-derived metrics to classify code fragments according to maintainability characteristics.

3.5 Summary

Research software is an imperative field for the development of human knowledge, allowing researchers to push forward and make meaningful contributions. Yet with this vital role as a tool, research software is often in an unmaintainable state due in part the lack of formal training that researchers have with 90% relying on self teaching (Hannay *et al.*, 2009) as well as the lack of emphasis put on the quality of software within the funding system (Howison and Herbsleb, 2014).

Static analysis is a method to bridge this gap, a method to raise the floor of software quality by using tooling to automatically flag maintainability risk. A core issue with this approach is the prevalence of false positives or "Ghost echoes"(Borg, Ezzouhri and Tornhill, 2024) which detract from using the tools (Johnson *et al.*, 2013), as experience is required to understand what is a true or false positive, experience which cannot be expected from researchers without formal training. A core reason for this is the reliance on universal expert rules which need to generalise across all domains and all projects, an approach which consequently produces false positives as maintainability from one domain to another will appear different.

The lack of a standardised testable method to determine maintainability is a key issue within the software engineering field which has had many different attempts to be solved, (Oman and Hagemeister, 1992) used numerical structural metrics to determine maintainability which suffered from "blind metrics" relying only on file level structural metrics. (Xiao *et al.*, 2016) took a more holistic approach to determine maintainability, analysing the historical data of the repository to calculate ATD hubs correlating to the highest impact files within a repository. This approach correlates to other studies showing how temporal data could be used as an alternative metric to identify ATD (Faragó, Hegedűs and Ferenc, 2015). These two studies informed the approach this study used to leverage temporal data to automatically create large scale maintainability datasets, leveraging this scale to circumvent the reliance on good commit hygiene to identify problem files that (Xiao *et al.*, 2016) required.

To properly take advantage of these large scale domain specific datasets a machine learning approach was used. It offers a compelling alternative to expert derived static analysis rules, by using interpretable models such as Decision Trees and Random Forests, human readable rules can be extracted that map directly to executable code quality checks. Random Forests provide higher accuracy but require additional effort to extract interpretable rules. Prior work has demonstrated strong classification performance (82-93% F1) (Schnappinger *et al.*, 2019; Bertrand, Favier and Andre, 2022; Bluemke and Borsukiewicz, 2023), yet

the persistent reliance on manually annotated, static datasets remains a bottleneck that prevents these tools from adapting to the evolving landscape of different domains.

This literature review has identified a gap in the current research landscape, while the need for maintainable research software is high, the tools available to achieve it are either too generic, too noisy, or too reliant on universal expert rules. The gap this study seeks to fill lies in the analysis of temporal analysis and inductive machine learning. By using version control metadata to automatically identify technical debt hotspots, we can generate a training set that is both objective and allows for empirical maintainability rules to be derived

4 Methodology

This study followed an empirical, data-driven pipeline to derive static analysis rules from real-world research software. The process was divided into four primary phases. First, we aggregated a dataset of C++ repositories from the Journal of Open Source Software (JOSS). Second, we employed a custom built tool to extract evolutionary coupling metrics and calculate a Hub Score for each file, providing an objective "High Risk" or "Low Risk" label. Third, these labeled files were used to train a Decision Tree and Random Forest model to identify the structural characteristics of high-risk code. Finally, the logical paths within the trained model were translated into human-readable static analysis rules, bridging the gap between historical developer behavior and proactive code quality standards.

4.1 Dataset Selection and Acquisition

The primary goal of this paper is to derive objective rules to raise the level of coding standards in the research software scene. This lends itself to using research software as the dataset, The Journal of Open Source Software (JOSS) (*Journal of Open Source Software*, no date) was selected as the primary source due to its peer-review requirement, which ensures a baseline of documentation and code quality.

4.1.1 Data collectiowere

A custom utility tool in rust was created to aggregate metadata from JOSS. This tool interfaced with Github API to gather repository-level metrics(contributor count, collaborator count). The resulting data was analysed by a python pipeline with pandas (team, 2020) to determine the final dataset. To ensure sufficient architectural complexity, projects were filtered based on the following criteria:

- Collaborator Threshold($>= 4$): Projects with fewer than four collaborators often reflect individual coding styles rather than standardized maintainability practices. A minimum of four contributors ensures a level of communication overhead that requires formal architectural patterns. As seen in Figure 1 the majority of repositories remain after this filtering, while low developer outliers are removed.

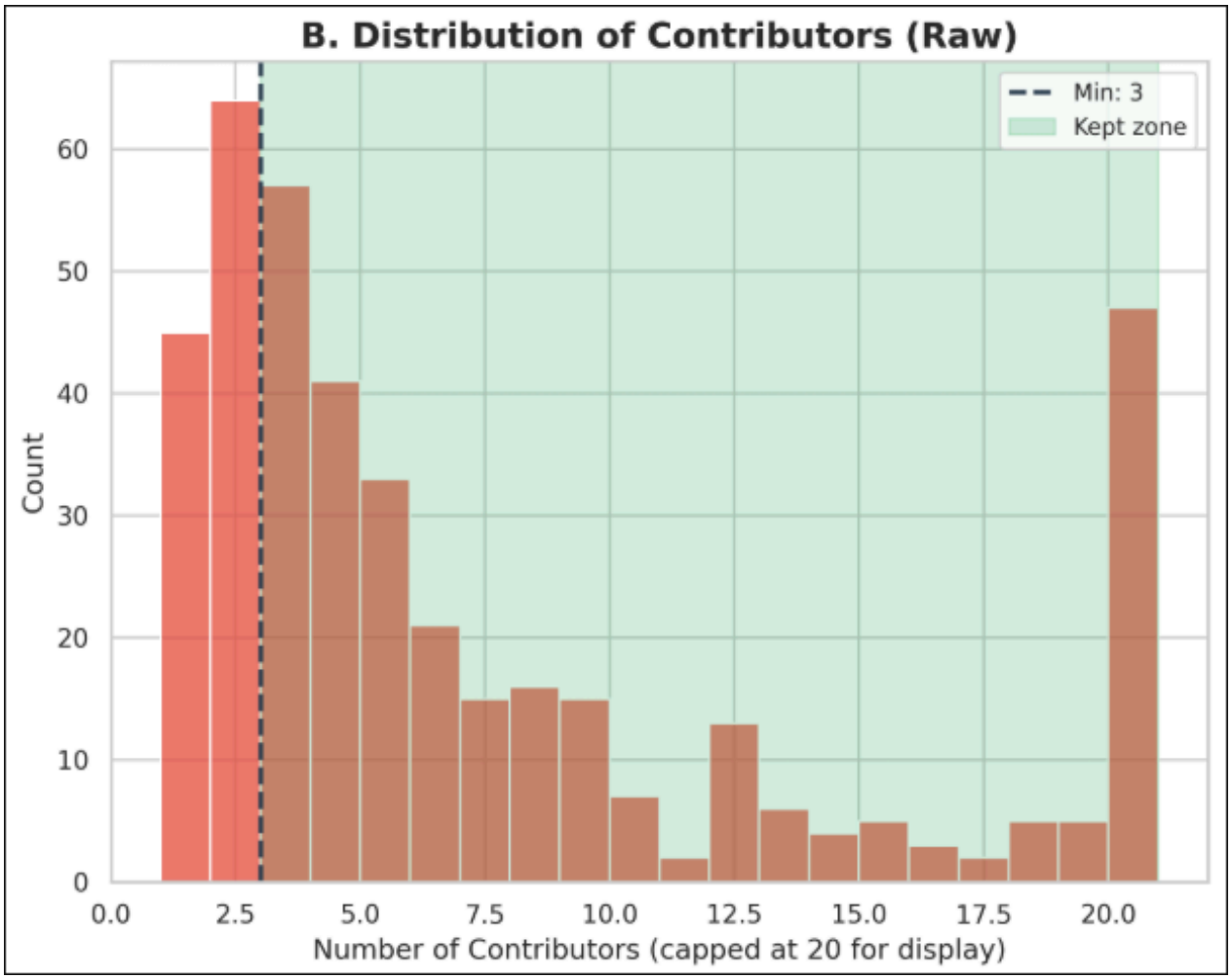

Figure 1: Pre filter Distribution of Collaborators

- Commit Volume (100–5,000): To ensure the dataset captures meaningful maintenance behavior, a "maturity floor" of 100 commits was established. Conversely, a "computational ceiling" of 5,000 commits was enforced to maintain feasibility on local hardware. As seen in Figure 2, these thresholds effectively remove the "short-tail" of immature projects and the "long-tail" of infrastructure-scale outliers. This dual sided filtering retains the vast majority of the JOSS population while ensuring a consistent and processable data scale.

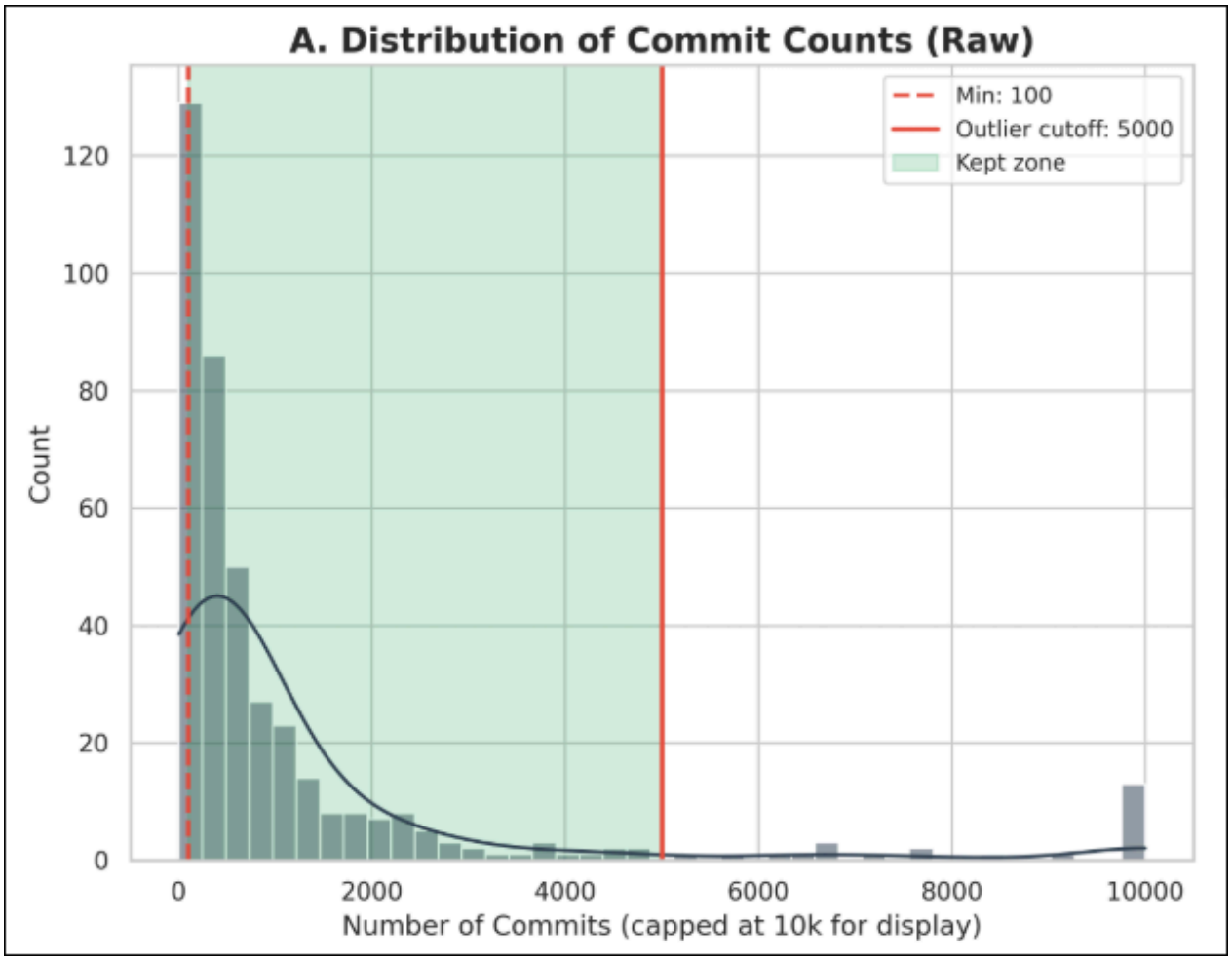

Figure 2: Pre filter Distribution of Commits

- Target Language: C++ was selected as the target language due to its dominance in high-performance research computing and the specific maintainability risks associated with its manual memory management and low-level abstractions. Restricting the study to a single language ensures methodological consistency, as code metrics are often non-comparable across different programming paradigms and syntax structures.

This filtering took the original 406 repos down to 213 which is a significant reduction in dataset size, notably those projects primarily consisted of individual small scale work that lacks the depth and collaboration overhead required to test maintainability work.

4.1.2 Dataset Co Change Graph Creation

To categorize the dataset, automated labeling heuristics were utilised rather than traditional expert manual review. While expert opinion is often the standard for establishing "ground truth", it is difficult to scale across hundreds of research repositories. Instead, the tool analyzed the historical behavior of each file to assign a Hub Score Section 4.2.1: a composite metric that measures a file's "dependency" within the project. This score is calculated based on three key factors: the number of coupled files, the

average coupling ratio, and the file's overall code churn. These metrics were gathered by running a custom analysis tool over the entire commit history of each repository. The tool constructs a Co-change Graph, where Nodes represent individual files and Edges represent shared commits between files. The graph allows us to evaluate the coupling of each file by comparing the number of times it was modified alongside other files versus the number of times it was modified in isolation.

To map the evolving relationships between source files a custom rust tool was created. Due to the scale of the relationships between files that needed to be tracked an in memory solution was not feasible, therefore Neo4j (Technology, 2015), was an ideal solution. As a graph database, Neo4j is uniquely suited for managing the complex, non-linear relationships inherent in software evolution. The rust tool performed a sequential traversal of each repository's entire commit history from the initial root, to the current head. At each commit tracking the codependency between each file as they were committed over time. This created a robust database of the temporal dependencies of the repository's architectures.

A key technical challenge arose over large commits. In research software researchers wouldn"t be aware of commit hygiene (Hannay *et al.*, 2009) leading to large scale "bulk" commits. If a dependency such as a dataset were uploaded of 1,000 files, that could generate 1,000,000 relationships to track. To maintain performance and ensure data relevance, a threshold was established to skip any singular commit containing more than 100 files. This optimization focused the dataset on intentional, developer-driven architectural changes rather than bulk file operations.

The output of the analysis pipeline was a Neo4j graph database, which served as the computational foundation for calculating file-level maintainability risks. By leveraging graph traversal queries, the dataset was categorized using a composite heuristic "Hub Score."

4.2 Dataset Labeling

Drawing on complementary insights from prior work, this paper proposes a composite Hub Score that combines multiple temporal metrics such as partner count and code churn into a singular composite numerical metric. (Xiao *et al.*, 2016) demonstrated that commit coefficients between files can serve as indicators of architectural debt. Files that frequently change together despite lacking structural dependencies represent "Modularity Violations" that represent ATD. (Faragó, Hegedűs and Ferenc, 2015) showed that files exhibiting high code churn (frequent line additions and deletions across commits) correlated directly with worse maintainability scores. (Cai *et al.*, 2025) empirically validated that coupling density measured by the number of partner files directly increases maintenance overhead across 1,200 Google repositories.

As noted by (Xiao *et al.*, 2016), focusing primarily on modularity violations ignores other potential risk factors. However, this approach is justified by the unique scale of the study. Unlike traditional, manually curated datasets such as PROMISE (Boetticher, Menzies and

Ostrand, 2007) or MainData (Schnappinger, Fietzke and Pretschner, 2020), the scale of the dataset allows for the most critical offenders across a significantly larger volume of data to be identified automatically. By combining the repositories analysed from JOSS (*Journal of Open Source Software*, no date), a dataset was created that is both objective and large in scope while providing a high-quality surface area for supervised learning.

To calculate a relative score of each file this formula was used to calculate a value called a Hub Score which represents the temporal coupling and change rate of every file in compared to others.

4.2.1 Hub Score

$$\text{HubScore} = |(C) \times \left(\frac{P}{F_t}\right) \times \left(\frac{W_f}{W_t}\right) \tag{2}$$

Variable Definitions

- $|(C)$ (**avg_coupling**): The mean strength or frequency of dependencies between this file and its partners.
- P (**partner_count**): The number of unique files that this specific file is coupled with.
- F_t (**total_files**): The total number of files in the repository or local subset excluding deleted files.
- W_f (**file_churn**): The number of changes or commits associated with this specific file.
- W_t (**total_churn**): The total sum of changes across the entire repository excluding churn from deleted files.

This is a bespoke formula normalising per file metrics proportionally to repository wide metrics to allow accurate comparison across repositories.

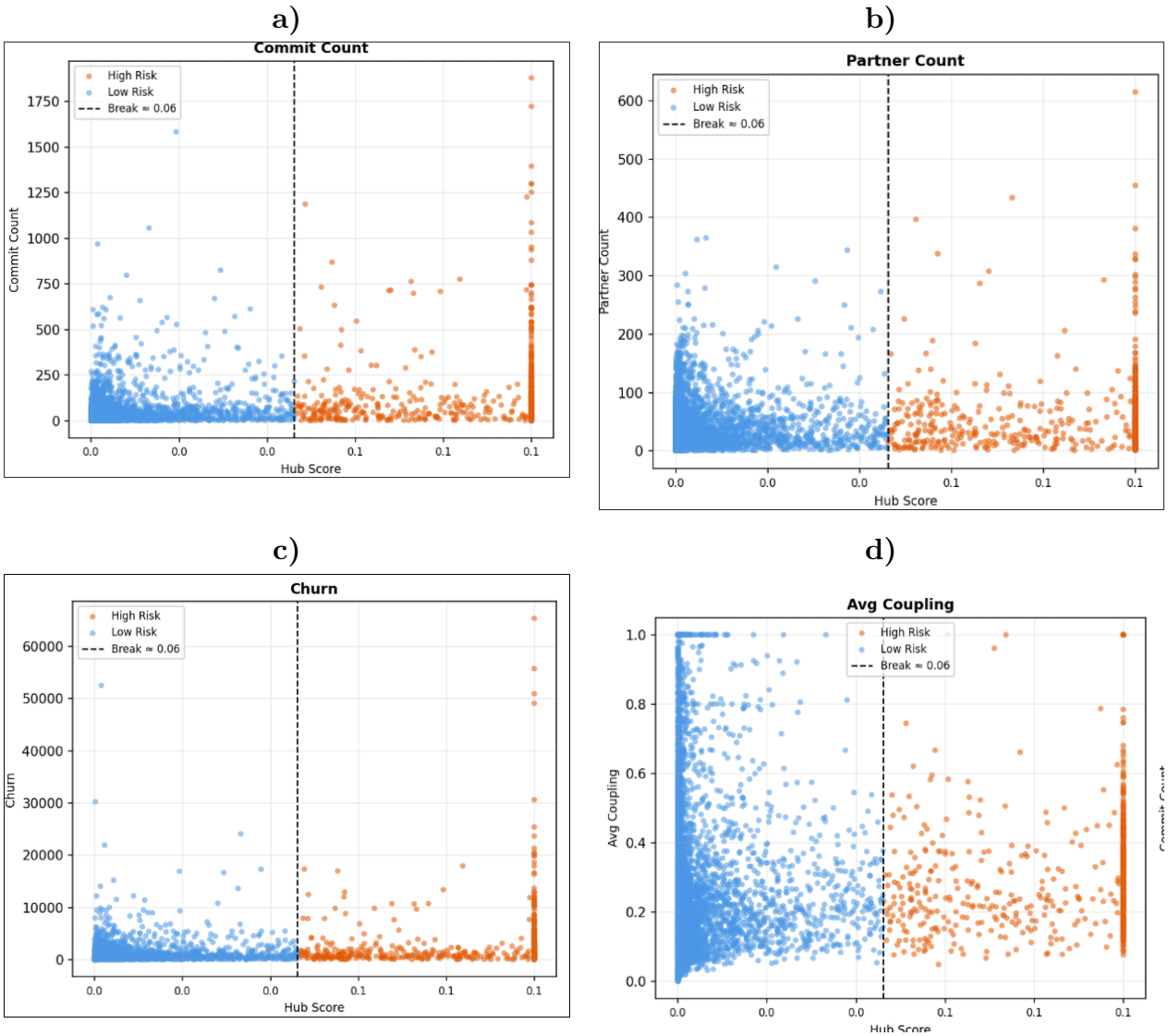

Figure 3: Hub Score Distribution across metrics: **a)** Commit Count, **b)** Partner Count, **c)** Code Churn, and **d)** Average Coupling.

The relationship between repository metrics and the Hub Score is visualized in Figure 3. While a positive correlation exists across all variables where higher values generally correlate to a higher Hub Score, the dispersion across each metric confirms that the formula successfully normalizes for scale and does not weight any one variable stronger than others. This prevents larger repositories with more history from automatically receiving high scores, ensuring that the identification of technical debt is context aware and comparable across repositories of varying scales. The need for the exclusion of deleted files is to avoid any potential bias towards smaller repositories as the normalisation methods would pull from a rich history that wouldn't accurately reflect the repositories current state.

The core justification to use this formula is derived from three core principles:

1. A logic file should not have to change often. Supported by the Open Closed principle (Meyer, 2009) and code churn analysis (Faragó, Hegedűs and Ferenc, 2015), which demonstrated that high frequency and large size of changes correlate to an increased maintenance overhead.
2. A logic file should not be highly coupled to many partner files. (Cai *et al.*, 2025) empirically validated that coupling density directly increases maintenance overhead across 1,200 Google repositories.
3. Files that change together despite lacking structural dependencies represent hidden architectural debt. (Xiao *et al.*, 2016) identified these "Modularity Violations" as the most common and expensive form of architectural debt, where files co-change without explicit dependencies.

This three principle justification is not merely theoretical; it mirrors the approach of prior work that uses evolution history as maintainability risk markers.This demonstrates that co-change information can produce high quality labels for maintainability risk detection as an alternative metric to expert annotation, validating the use of temporal coupling as an objective labeling mechanism for machine learning rather than hotspot detection alone.

4.2.2 File classification and Oversampling

To prepare the data for a binary Decision Tree classifier, the continuous Hub Score was split into two categorical classes: High Risk and Low Risk. Rather than applying an arbitrary numerical threshold, Jenks Natural Breaks algorithm was employed to identify the ideal split point (Jenks, 1967). This algorithm identifies natural groupings within the data by minimizing the variance within each class while maximizing the statistical separation between the classes. Ensuring that high risk labels are assigned to architectural outliers rather than files marginally exceeding a mean or median value. To mitigate the risk in overcompensating for outliers the algorithm breaks the Hub Score into 3 classes (k=3) and the breakpoint from the middle class was selected as the threshold for high risk.

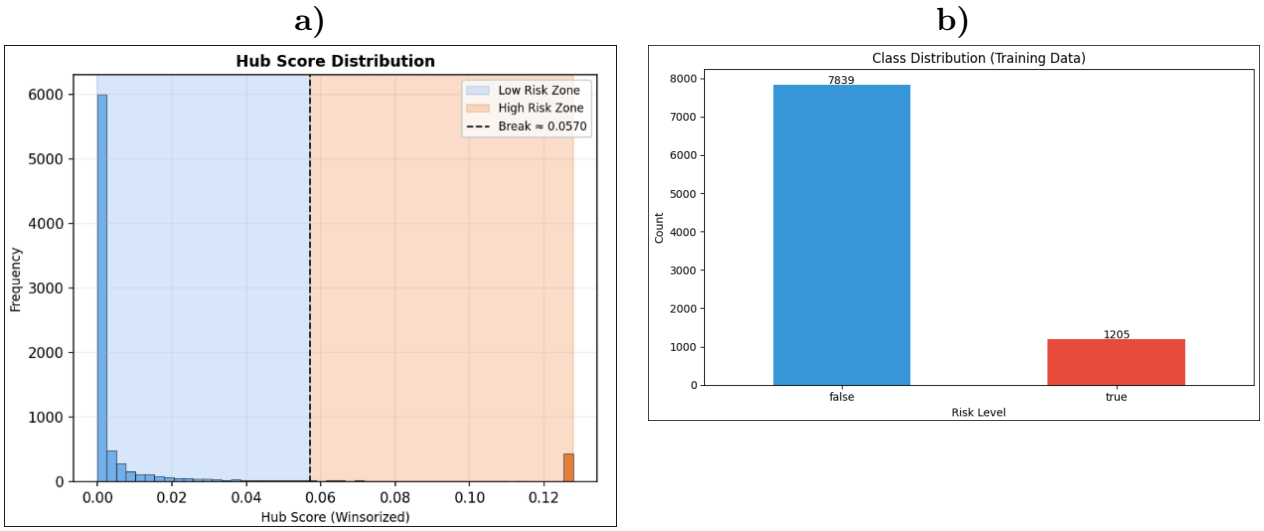

Figure 4: Hub Score Distribution: a)Hub Score distribution across all .cpp samples , b) Distribution of high and low risk files

As normally observed in software the distribution of architectural debt is often skewed Figure 4, with a small number of files contributing to the majority of risk and maintenance overhead, the same separation was observed within this dataset Table 1 with a 6.5:1 ratio of high to low risk files being observed.

Total	**Post Cleaning**	Jenks LR	Jenks HR
9,060	9,044	7,839	1205

Table 1: Dataset file counts at each step

With a disparity this large between labels and the need for a dataset when performing binary classification to avoid a majority class bias towards low risk files, Synthetic Minority Over-sampling Technique (SMOTE) was implemented. Following the precedent for using SMOTE set by (Bluemke and Borsukiewicz, 2023) in maintainability prediction, SMOTE generates synthetic observations within the feature space rather than simply duplicating existing records (Chawla *et al.*, 2002). Providing the model with a balanced training environment and improving the recall of high-risk files without compromising the classifier's ability to identify maintainable code.

4.2.3 Dataset Verification Method

To address the validity of the autonomous labeling process (RQ1), a verification step was performed to reveal the extent Hub Score aligns with established architectural risk benchmarks.

To test this CodeScene was used as a ground truth for architectural technical debt, CodeScene categorizes file health into three tiers: Healthy (Health > 9), Problematic (Health 4–9), and Unhealthy (Health < 4). Following the definition established by (Borg,

Ezzouhri and Tornhill, 2024), both 'Problematic' and 'Unhealthy' classifications were aggregated into a single 'High Risk' label for the purpose of this study.

The verification process involved a set of repositories from CERN (*Home | CERN*, no date) used to calculate industry standard labels for high risk for architectural debt per file. A custom extraction tool was created to retrieve architectural data from the codescene.io which was then subsequently put into standardised CSV structure for comparison.

To generate the corresponding Hub Score labels, each repository was assessed individually. A static threshold was derived for each repository using the Jenks Natural Breaks algorithm. This allowed for the autonomous separation of files into binary classes, High and Low Risk, based on their specific Hub Score.

These were then cross-referenced against the CodeScene derived labels for a final comparison with the primary metric used to determine the similarity of the datasets being Cohen's Kappa (Cohen, 1960). This method is more robust than a simple accuracy check as it accounts for the possibility of agreement occurring by pure chance, which is a key consideration with the imbalanced ratio seen in Figure 4 of low to high risk. By using Kappa, the study provides a more thorough evaluation of how the Hub Score method aligns with established static analysis, moving beyond surface level accuracy to reveal the true reliability of temporal metrics in identifying maintainability risks, fully assessing the extent of similarity between both datasets showing the efficacy of the Hub Score method.

4.2.4 Addressing Data Leakage

A critical consideration in this methodology is the separation of labeling criteria and training features. The labels are derived from temporal metadata (commit history, co-change frequencies, and historical churn). The features (inputs for the Decision Tree and Random Forest) are derived strictly from static source code analysis (e.g., McCabe Complexity, LoC, Nesting Depth). This separation ensured that the model is learning to predict future maintenance risk based on the current state of the code, rather than simply "re-discovering" the components of the Hub Score formula.

4.2.5 Dataset Summary

The final dataset consists of 9,000 labeled files from 200+ research repositories distributed across each repository Figure 5.

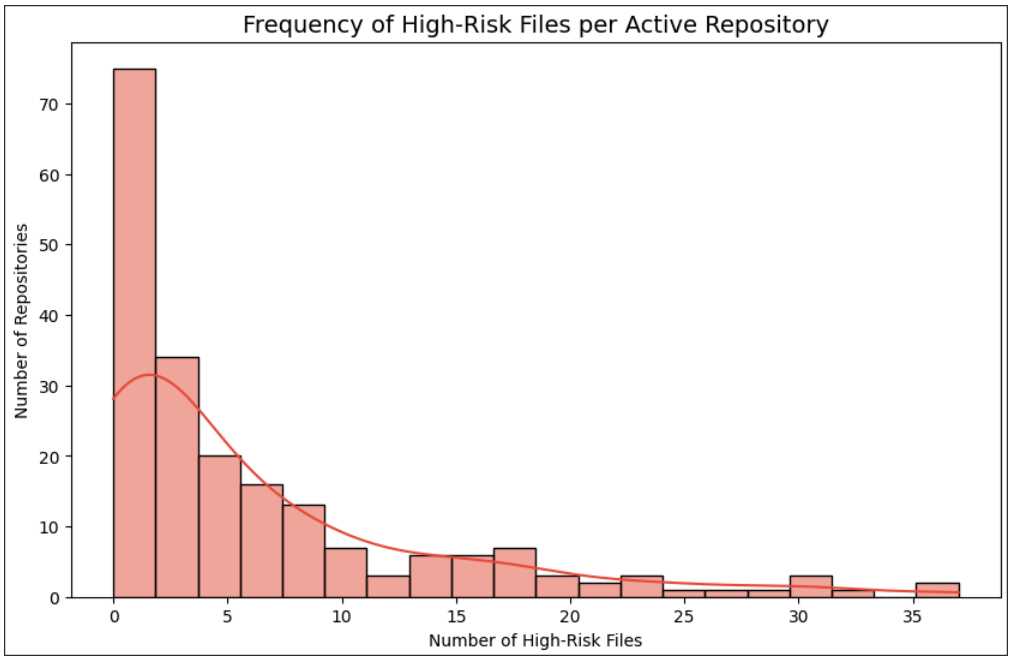

Figure 5: High risk distribution across all repositories

The dataset was generated entirely automatically using the custom rust and python tooling using the empirically defined method. There is a strong class imbalance between high and low risk with a majority bias towards high risk Figure 4 requiring the models to learn structural relationships to accurately label a file as high risk.

4.3 Feature Engineering

To prepare the high risk and low risk datasets for machine learning, the raw source code files had to be converted into tabular data compatible with Decision Trees and Random Forests. Feature extraction was performed using the rust-code-analysis (RCA) crate (Ardito *et al.*, 2020). RCA was used on the base source files to statically derive numerical metrics on the files, the metrics can be seen in Table 2.

There is a notable risk of falling into the pitfall of using "blind metrics" like in other methods, a key differentiator here is that the high risk labels have been derived entirely from temporal data. By utilizing file-level structural metrics to predict repository wide temporal outcomes, the model is incentivized to identify holistic architectural patterns. This ensured the model learns the structural signatures of actual maintenance debt rather than merely flagging localized code smells.

Metric	Description
CC	McCabe's cyclomatic complexity: it calculates the code complexity examining the control flow of a program. It is measured as the number of linearly independent paths through a piece of code.
SLOC	Source lines of code: it returns the total number of lines in a source file.
PLOC	Physical lines of code: it returns the total number of instruction and comment lines in a source file.
LLOC	Logical lines of code: it returns the total number of logical lines (statements) in a source file.
CLOC	Comment lines of code: it returns the total number of comment lines in a file.
BLANK	Blank lines: it counts the number of blank lines in a source file.
Halstead	The Halstead suite, a set of seven statically computed metrics, all based on the number of distinct operators (n1) and operands (n2) and the total number of operators (N1) and operands (N2). The suite provides a series of information, such as the effort required to maintain the analyzed code, the size in bits to store the program, the difficulty to understand the code, an estimate of the number of bugs present in the codebase, and an estimate of the time needed to implement the software.
MI	Maintainability index: a suite to measure software's maintainability, calculated both on source files and functions.
NOM	Number of Methods: it returns the number of methods in a source file.
NARGS	Number of arguments: it counts the number of arguments of each method in a source file.
NEXITS	Number of exits: it counts the number of possible exit points of each method in a source file.

Table 2: Extracted code metrics (Ardito *et al.*, 2020)

Prior to model training the extracted features underwent additional preprocessing to enhance performance. Columns containing NaN values were removed, as certain language-specific metrics were non-applicable to the C++ source files within the dataset. The Maintainability Index (MI) column was dropped to remove potential bias from previous maintainability prediction methods. Low variance (<0.01) and highly correlated features ($|r| < 0.9$) were dropped to reduce the search space when selecting variables. Simple ratios between relevant features (Coleman *et al.*, 1994) were computed such as operator operand ratio to capture relative relationships between measurements. Lastly due to the nature of tree based machine learning models further metrics were calculated using sklearn's (Pedregosa *et al.*, 2011), polynomial feature generation on key numerical metrics to allow non linear metrics to be an option for the tree model feature selection.

4.4 Model Implementation and Training

Following the refinement of the feature set, the processed tabular data was utilized to train and validate two supervised learning models: a Decision Tree and a Random Forest. These models were selected for their inherent interpretability, allowing the extracted maintainability metrics to be mapped to clear, logical decision boundaries.

To ensure a robust evaluation of both models, a standardised training approach was used. Sklearns's (Pedregosa *et al.*, 2011), DecisionTreeClassifier and RandomForestClassifier were used as the base models. GridSearchCV was used to perform hyperparameter fine tuning on the models where F1 score was used as the primary optimisation target ensuring a balanced performance metric between high and low risk files. A 5-fold stratified cross-validation was used over a traditional train-test split to ensure that each fold remained representative of the overall class distribution, mitigating the risk of sampling bias that can occur with a single split.

Coupled with this to aid in the recall of the model to lower the chance of a false negative, a weighted cost function was applied, penalizing the misclassification of high-risk files more heavily than low-risk files.

The optimal configurations for decision trees and random forests are detailed in the results section. Table 4

4.5 Rule Derivation Process

The selection of tree-based models was driven by the requirement for rule extraction. Once the models reached optimal performance, two distinct methodologies were used to translate the mathematical weights into human-readable coding standards:

- Decision Trees: Decision Trees provide an inherent "IF-THEN" logical structure. To ensure interpretability, the models were constrained by maximum depth parameters during training. The trained model was exported as a visual representation using scikit learns's tree visualization tools (Pedregosa *et al.*, 2011), allowing for a clear view of the decision paths.
- Random Forests: Due to the complex ensemble nature of random forests only a derivation of the rules could be extracted (Haddouchi and Berrado, 2025). Using the tool te2rules (Lal, Chen and Mithal, 2024), an approximation of the random forests rules could be generated to a high fidelity providing a human-readable summary of the ensemble's collective logic.

The extraction of human readable rules is a strong step towards practical application. However they cannot be trusted without empirical testing as without it they cannot be trusted over expert rules.

4.6 Evaluation Framework

To determine the validity of the ML-derived rules testing against the current Sota in static analysis is required, for which CodeScene was the tool selected. Performance was benchmarked against a set of repositories selected from the CERN (*Home | CERN*, no date) software ecosystem that fit the same rigorous criteria as the primary dataset. CERN was selected as it is a pillar of the research software community, providing critical infrastructure used by thousands of researchers globally. Performing an analysis on these repositories removes the potential for "cherry-picking" by testing the framework on mature, high-stakes codebases that were not part of the initial training phase.

To test the models' utility in real-world scenarios, an external validation was conducted using a series of CERN repositories held separate from the training set. For these CERN repositories, industry-standard analysis from CodeScene was used as the ground truth. The random forest model was run against these ground truth labels to derive an F1 score to assess the application of the models in the wider research software development scene.

4.7 Threats to Validity

This subsections highlights the every potential path identified that could bias the results within the methodology of this study.

4.7.1 Construct Validity

Construct validity examines whether the theoretical constructs (e.g., "Architectural Decay") are accurately measured by the chosen metrics (the Hub Score).

- Temporal Proxy Limitations: While the Hub Score has been observed to act as a measure for maintainability risk in files, it can only act as a proxy for measurement. This is because a file can exhibit high churn due to non architectural reasons. The composite nature of Hub Score does mitigate this; there is still a risk for files marked as high-risk to be low risk and low risk files being misclassified as high risk by Hub Score.
- Expert Rule Alignment: This study has argued that domains require specific rules while simultaneously using expert derived universal rules as "ground truth". This study argues that Hub Score is a more empirical measurement; it is worth noting that without an agreed upon method of measuring maintainability, Hub Score is only one of many approaches for measuring maintainability.

4.7.2 Internal Validity

Internal validity relates to factors within the experimental design that may have biased the results.

- Computational Constraints: Due to the use of a single execution environment the analysis of commit history was capped in both commit count and number of changes per commit to ensure feasible processing times. This may have led to "deep history" of older repositories being missed that could've influenced the overall Hub Score distribution.

4.7.3 External Validity

External validity concerns the generalisability of the findings to other contexts.

- Benchmarking Comparison: Due to constraints of using the CodeScene client, large scale analysis of repositories was unavailable, this led to a sampled benchmarking approach to be used for external verification. This sample followed the same criterion as the repositories in the dataset but may yet have been biased for or against the study.

4.8 Summary

The methodology presented here establishes a complete pipeline for empirically deriving maintainable, opinionated static analysis rules from software evolution data. Leveraging the scale of 200+ different research repositories to generate a dataset using objective metrics by combining temporal coupling analysis with structural metrics. Then taking this dataset using machine learning approaches to produce interpretable accurate rules that reflect actual developer behavior rather than relying on universal expert rules.

5 Results

This section will present the performance of the data labelling performed using the Hub Score metric, the F1 score of the models trained on the dataset, the efficacy of the extracted rules and the comparison to current static analysis tooling.

5.1 Hub Score Comparison with CodeScene

Looking at RQ1, to accurately assess the extent that version control metadata can autonomously label architectural risk in domain specific repositories a benchmarking against 5 CERN repositories was performed. The code health metrics from CodeScene when using the code health < 9.0 set out from (Borg, Ezzouhri and Tornhill, 2024) when compared to the Hub Score metrics led to an insightful conclusion.As revealed in Table 3, the comparison yielded an average Cohen's Kappa ratio of 0.224. When evaluated against the benchmarks established by (Landis and Koch, 1977),this indicates a 'Fair' strength of agreement. This result demonstrates that the Hub Score composite metric can emulate the structural identification criterion using only temporal metrics as to extent which is viable. This level of agreement suggests an intersection between expert structural rules and temporal evolution, however the discrepancy between CodeScene and Hub Score is a sign that while there is an overlap between expert structural rules and temporal metrics yet they remain distinct enough to each capture different dimensions of maintainability.

	ACTS	LGC2	Vecmem	SixTrackLib	Merlin++	Average
Hub HR, CodeScene HR	20	37	6	49	18	
Hub LR, CodeScene LR	690	16	56	51	67	
Hub HR, CodeScene LR	8	0	6	34	20	
Hub LR CodeScene HR	152	36	16	18	28	
Total Files	876	89	84	152	133	
Accuracy %	81.6%	59.6%	73.8%	65.8%	63.9%	70.94%
Cohen's Kappa	0.15	0.27	0.21	0.32	0.17	0.224

Table 3: High risk identification comparison Hub Score and CodeScene

5.2 Machine Learning Models

5.2.1 Model Configuration

To ensure the results were not the product of sub optimal configuration, GridSearchCV was employed to identify the optimal parameters using F1 score as the objective function. This was conducted on both the decision tree and random forest models and the final configuration can be seen here Table 4

Parameter	**DT Value**	**RF Value**
Class Weight	False: 1, True: 2	False: 1, True: 3
Criterion	entropy	N/A
Max Depth	7	30
Max Features	N/A	sqrt
Min Samples Leaf	5	4
Min Samples Split	10	10
N Estimators	N/A	300

Table 4: Model hyperparameters

5.2.2 Model Performance

Using the optimal configuration in Table 4 5 fold stratified cross validation on the SMOTE-balanced training dataset was conducted.

Model	Accuracy	Precision	Recall	F1
Decision Tree	69.5%	24.7%	60%	35%
Random Forest	81.2%	36.1%	49.1%	41.6%
SMOTE Decision Tree	71.2%	25.4%	56.7%	35.1%
SMOTE Random Forest	81.1%	36.7%	52.9%	43.3%

Table 5: Model Performance Comparison

Table 5 reveals that both random forest an decision tree models marginally benefited from SMOTE, due to the imbalance of the dataset noted in Figure 4 this is a surprising result. The overall F1 score of every model with 43.3% being the highest, accuracy was the highest score for each model and this aligns with the dataset imbalance rewarding models for being more conservative as 85% of values are low risk. Due to the almost identical performance of SMOTE and raw models, only SMOTE models will be discussed from this point onwards.

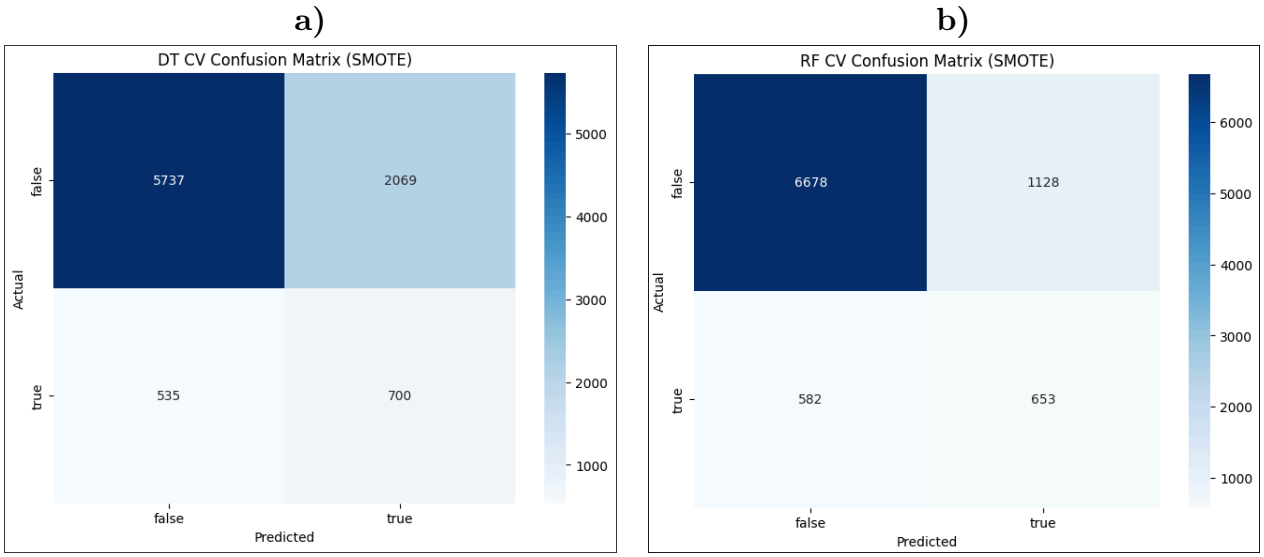

Figure 6: Confusion Matrices of predicted metrics: **a)** Decision Tree SMOTE, **b)** Random Forest SMOTE,

Figure 6 reinforces the divergent strategies of the two models. The Decision Tree demonstrates a 'Hyper aggressive' strategy maintaining accuracy and recall by sacrificing precision, predicting high risk a disproportionately high amount of the time, higher even than the total number of high risk samples in the dataset were incorrectly predicted with 2769 predicted high risk while there are only 1205 high risk samples in the entire dataset Table 1. Conversely, Random Forests demonstrates a more 'Conservative' strategy leveraging the additional predictive power of the ensemble of trees, it achieved a much higher recall by more accurately being able to predict high risk files achieving a precision of 36.7% in comparison to the Decision Tree's 25.4%. As evidenced by Figure 6, both models produced a higher volume of False Positives than True Positives. For the Random Forest, the precision of 36.7% indicates that for every 10 files flagged as 'High Risk,' approximately 6 to 7 are likely stable files. This 'Ghost Echo' effect remains a significant barrier to developer adoption, as the tool effectively generates more 'noise' than actionable 'signals' (Johnson *et al.*, 2013)."

5.2.3 Feature Importance

To understand the underlying logic of the classifiers, the feature importance was extracted from the Decision Tree and Random Forest model. This identifies which static metrics provided the strongest signal when predicting the temporal Hub Score.

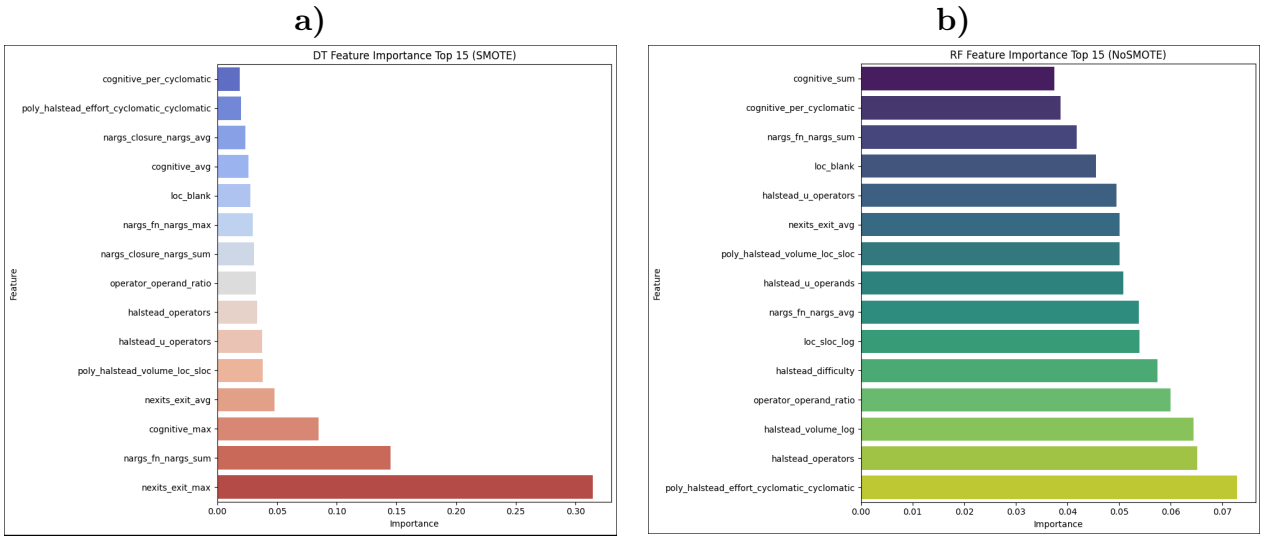

Figure 7: Confusion Matrices of predicted metrics: **a)** Decision Tree Feature Importance , **b)** Random Forest Feature Importance,

As shown in Figure 7, the Decision Tree model demonstrates a highly skewed importance distribution, with a sharp decline in predictive weight following the primary feature. This 'top-heavy' profile is characteristic of the Decision Tree's greedy splitting approach, where the root node (the most discriminative feature) captures the majority of the information gain. The model primarily relies on NEXITS, followed closely by NARGS. This indicates that the tree prioritizes the number of exits connections a file has as well as the number of potential cases a function must handle were the most valuable metrics when identifying architectural hubs. Interestingly, the raw metrics were prioritised for the Decision Tree over the polynomial metrics generated suggesting that structural base metrics provided a clearer picture for Hub Score rather than complex feature interactions.

When evaluating the Random Forest model a very different distribution, as the ensemble nature allows many trees to have the same variables as such the distribution of values is closer together. Halstead metrics are by far the most popular metric used with 5 out of the first 6 including Halstead metrics. These metrics, which quantify program volume, vocabulary, and effort based on operator and operand counts, appear to provide the classifier with a more nuanced picture for identifying temporal hubs. This suggests that for the Random Forest, the density and variety of operations within a file are more indicative of evolutionary risk than the NARGS and NEXITS.

5.2.4 Extracted rules

The rules were extracted using a visualisation of the Decision Tree and using te2rules (Lal, Chen and Mithal, 2024) on the Random Forest.

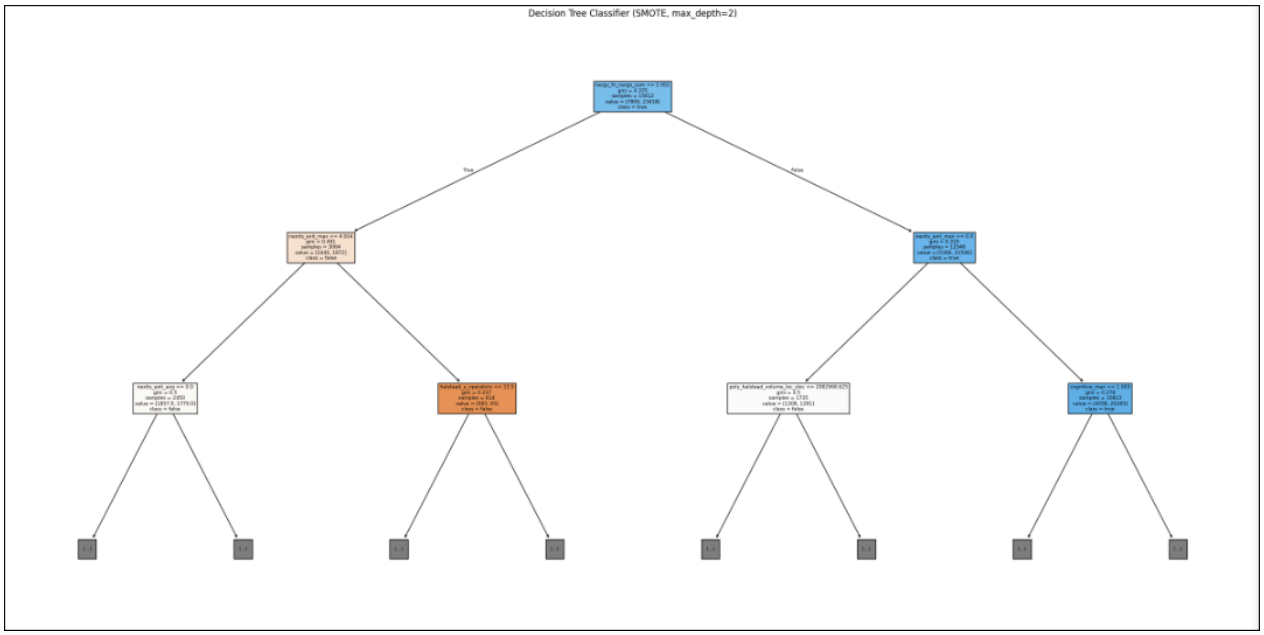

Figure 8: Decision Tree Visual representation

Figure 8 is a high level snapshot only showing the highest level of the 15 deep Decision Tree for readability. Revealing the simple rules that the decision tree uses such as nargs_fn_nargs_sum <= 2 or nexits_exit_max <= 4.914, highlighting the simplicity of the Decision Tree model at the cost of F1.

```
cognitive_avg > 3.502742886543274
& cognitive_max > 8.72629690170288
& cognitive_per_cyclomatic <= 3.3673081398010254
& cognitive_sum > 1.0019041299819946
& halstead_difficulty > 105.43690872192383
& halstead_u_operands <= 403.5
& halstead_u_operators <= 40.99243354797363
& loc_blank <= 425.5975036621094
& loc_blank > 59.552019119262695
& loc_sloc_log > 6.696109294891357
& nargs_fn_nargs_avg <= 25.373714447021484
& nargs_fn_nargs_sum > 2.0009937286376953
& nexits_exit_avg <= 6.298295497894287
& nexits_exit_sum <= 19.99100399017334
& operator_operand_ratio <= 1.535413682460785
& poly_halstead_effort_cyclomatic_cyclomatic > 6914969.5
& poly_halstead_volume_loc_sloc <= 115728892.0
```

Logic 1: Example of one of the top 5 rules for determining maintainability derived by te2rules

A clear inverse relationship exists between the predictive power of Decision Trees and the performance of Random Forests. The logic generated by te2rules from the Random Forest model was entirely non-actionable for human stakeholders; each of the top five most significant rules consisted of at least 15 distinct terms. Logic 1 shows that these convoluted

and overly complex rules create an entirely unreadable set of requirements. Such rules offers no clear path for developers to drive impactful refactoring changes, the thresholds are too granular and unintuitive to be applied in a human developer context.

Building on this, when coverage was evaluated (the number of files matches correctly by this rule over the total number of files being assessed) for the top 5 performing rules derived from the Random Forest, the cumulative coverage of the 5 rules was 2.45%, an abysmal result.

A probable explanation for this result is the low signal to noise ratio within the feature set, which forces the model to construct complex groups of weak indicators to achieve any predictive power, leading to overfitting to specific sections of the dataset. The model's inability to identify generalisable patterns left te2rules with only highly complex rules tree ensembles to derive logic from. Coupling this with the granular nature of the feature set, which improved the options for the model, the model opted for polynomial features such as `poly_halstead_effort_loc_sloc`, which are practically incomprehensible for a human developer.

5.2.5 External Validation

External validation was conducted on a suite of repositories from CERN.These repositories were entirely excluded from the training phase to test the generalisability of the Random Forest model against CodeScene's ground truth labels following the standard from (Borg, Ezzouhri and Tornhill, 2024).

	ACTS	LGC2	Vecmem	SixTrackLib	Merlin++	Average
RF HR, CodeScene HR	119	26	4	35	17	
RF LR, CodeScene LR	587	44	71	63	175	
RF HR, CodeScene LR	54	11	8	48	25	
RF LR CodeScene HR	112	1	6	18	8	
Total Files	872	89	84	152	225	
Accuracy %	81.0%	78.7%	89.3%	64.5%	85.3%	79.76%
Cohen's Kappa	0.47	0.7	0.27	0.2	0.43	0.414

Table 6: High risk identification comparison SMOTE Random Forest and CodeScene

Table 6 reveals that the model improved in its predictive ability at a greater rate than the ground truth labels with an average kappa score increasing from 0.224 to 0.414. This shift from 'Fair' to 'Moderate' agreement (Landis and Koch, 1977) over the baseline Hub Score analysis Table 3 serves as evidence that the Random Forest is not merely regressing toward the majority class (Low Risk). Instead, the model is successfully identifying the latent structural criteria employed by CodeScene.

The doubling of the Kappa coefficient, a metric the explicitly punishes chance agreement, suggests that the model has learned a domain specific structural classifier through temporal

metadata labels all while remaining only on file level structural metrics. This confirms that domain specific empirical maintainability datasets can provide machine learning methods the ability to learn generalisable rules for predicting maintainability risk across a domain. By maintaining a Moderate correlation to Sota approaches with universal rules, this method demonstrates the ability to identify what rules correlate to each domain and the ability to identify new unseen domain specific rules. Effectively, the model is seen to be able to bridge the gap between temporal metadata and structural metrics, proving that specialised temporal training can yield similar predictive utility as Sota expert derived maintainability risk indicators within the target domain.

5.3 Summary of Findings

5.3.1 Autonomous Labeling Accuracy (RQ1)

Using Hub Score a viable empirical alternative to the current Sota in expert rules (CodeScene) was verified. Functioning as a method to evaluate, the Hub Score demonstrated a correlation with structural architectural vulnerabilities while remaining distinct enough to be used maintainability dataset. This confirms that temporal metadata can be used autonomously label high risk files, providing a scalable efficient labeling mechanism for domain specific maintainability research.

5.3.2 Opinionated Interpretable Guardrail Rule Derivation (RQ2)

Random forest with an F1 of (43%) outperformed Decision Trees (35%), the accuracy for both remained high due to the class imbalance in the final training set Figure 4 but both models failed on achieving a high precision or recall. This meant that many false positives were present particularly within decision trees, undermining the efficacy of the derived guardrails due to the uncertainty in predictions.

However, the external validation reveals a significant finding: the model's agreement with CodeScene's ground truth nearly doubled, with the average Cohen's Kappa increasing from 0.224 to 0.414. This shift from "Fair" to "Moderate" agreement suggests that the model successfully learned a domain specific structural classifier in line with some of the current Sota expert rules using only temporal metadata labels. This confirms that the Hub Score is not a random random, but a measurable indicator for structural state that machine learning can detect, effectively bridging the gap between historical evolution and static code metrics.

On the other hand, the interpretability of the resulting rules yielded mixed results. While the models can classify risk with high accuracy, the logic required to reach those classifications is too complex for human developers. The derived rules are either too granular to be generalizable or rely on abstract composite metrics (Seen here Figure 8) that offer no clear refactoring path for a developer.

Consequently, this study validated that empirically derived static analysis rules agnostic to the current landscape of expert rules through ML to predict historical maintainability

risk is a viable approach. The complexity of architectural decay in research software resists simple heuristics suggesting that the value of these models lies not in black box quality gates over deriving a human alternative to expert derived rules.

6 Discussion

6.1 Evolutionary Risk and Structural Metrics(RQ1)

When examining the results from Table 3 it is clear that the Hub Score has a correlation with the high risk files identified structurally by CodeScene. While there is a noticeable discrepancy between the Hub Score and CodeScene predictions with a Kappa score of 0.224 indicating a 'Fair' strength of agreement (Landis and Koch, 1977). This level of agreement using only temporal metadata, is a significant enough to justify that evolutionary risks can be used as an alternative maintainability dataset which has a majority overlap with expert defined rules. The discrepancy between the two metrics suggests that the Hub Score captures a distinct dimension of maintainability that overlaps with, yet remains separate from, traditional static analysis providing unique insights in an empirical manner.

6.1.1 Temporal Metrics as a Proxy for Structural Decay

As the Hub Score is entirely derived from temporal metrics, it reveals a correlation between historical metadata and the structural expert rules that are currently the standard across the field. Similarly to what (Faragó, Hegedűs and Ferenc, 2015) identified when observing the impact of code churn on maintainability, a composite representation of temporal metadata is shown to be correlated to the current industry standard structural metrics.

Historical metrics can and should be used in tandem within static analysis to not only identify refactoring targets as CodeScene currently does, both structural and temporal metrics should be used as a metric for identifying high risk files. This could supplement current refactoring methods by highlighting problems without a clear structural issue that would normally go undetected.

6.1.2 Implications for Large-Scale Research

Table 3 validates the use of analytical methods based on temporal metrics as a scalable alternative manual expert defined rules. By circumventing the need for proprietary tooling or manual expert intervention, this methodology enables empirical studies of software health at scale in domain specific areas.

6.2 The Structural-Temporal Mismatch (RQ2)

As observed in Table 5, a significant disparity exists between the high accuracy (71% - 82%) of the models and their relatively low F1 scores (35%–43%). This gap reveals that structural metrics, which provide a stateless snapshot of a file, are insufficient sole predictors of the stateful, temporal risk captured by the Hub Score.

Logically this mismatch is expected, there may be a weak correlation between temporal changes and structural metrics but there are too many examples which serve as counterexamples to this being a strong correlation: A god file so large people are afraid to interact with, that has no new changes is labelled as low risk by Hub Score and high

risk structurally, a heavily coupled but well designed file that changes every version is labelled as high risk by Hub Score and low risk structurally. Each of these examples show how viewing code through one lens be it structurally or temporally leaves analysis open to architectural blind spots which should not be ignored if raising code quality is the goal.

The strides made by (Xiao *et al.*, 2016) identified that temporal analysis alone could identify maintainability hotspots with the right repository commit hygiene. The results here indicate that when handling repositories where commit hygiene is not being maintained, a hybrid approach is necessary one, where a machine learning model trained on temporal and structural metrics in tandem has the potential to increase the accuracy in predicting maintainability labels.

The measurable shift from a 0.224 Kappa in the raw Hub Score to a 0.414 Kappa in the Random Forest validation Table 6 is particularly insightful. This "Moderate" agreement proves that the machine learning model is not merely replicating the majority class bias but is learning structural criterion that correlate to Sota expert rules outperforming the dataset it was trained on. By bridging this gap, the model demonstrates that specialized temporal training can yield predictive results comparable to expert derived rules within the target domain, confirming that temporal metrics can guide machine learning models to empirically identify structural maintainability issues for a target domain.

Moving forward, while this study has shown that file-level metrics alone contain insufficient signal for perfect prediction, the successful extraction of a "Moderate" structural-temporal correlation marks a significant step toward autonomous machine learning static analysis. The focus moving forward should be on file level metrics for prediction should move to a platform level metric position, identifying high risk files by assessing its place in the overall repository is a clear path forward. This research proves that while the correlation between a singular file's structure and its historical risk is complex, the ability of ML to bridge these two domains Table 6 provides a robust foundation for models to accurately predict maintainability labels based on structural metrics through an empirical temporal lens.

6.2.1 Confidence in Actionable Guardrails

The choice in using file level metrics was made to facilitate the generation of human-readable, interpretable rules. However, the results suggest a fundamental disconnect between machine-learned patterns and actionable developer guardrails.

The core idea of the interpretable rules using only structural metrics was to allow for refactoring goals to be created with those in mind. High dimensional structural metrics such as `poly_halstead_effort_cyclocmatic_complexity` Figure 7 provide the models with the tools required to reach their current F1 score which is still quite low. They are fundamentally different to how expert rules are created to target areas for refactoring. The rules derived from these models were either too granular (overfitted to specific files) or relied on abstract composite metrics that offer little guidance for refactoring.

For rules to be actionable it must provide a clear target a developer can understand and fix. As the machine learning models prioritise statistical probability over developer comprehensibility, the resulting lack of rules clarity is a side effect of attempting to derive comprehensible rules from statistical machines.

6.3 What this means for Research Software (RQ2)

6.3.1 The Empirical Ground Truth

The capabilities of Hub Score file classification mark a path for researchers to conduct empirical analysis on code maintainability without relying on preestablished generalised expert rules or hand annotated data. Allowing researchers to quickly and efficiently analyse large amounts of repositories to conduct domain specific or large scale analysis with minimal compute. This provides a standardised approach to serve as a foundation for future maintainability studies.

6.3.2 The Changing Landscape from Humans to AI

The focus on ensuring the models could generate human interpretable rules through the use of Decision Trees and Random Forests limited the capabilities of the models used. As there is a paradigm shift in how software is being made and the current research landscape will begin using generative AI more in the same way the entire software engineering industry is moving.

If that is the case, should the focus be on the understandable and interpretable refactoring rules? Should researchers spend time, effort and money on learning how to write software to the same standard as a full time software engineer?

Researchers should be able to focus on their research and tools should guarantee the floor of software quality automatically to avoid issues such as the reproducibility problems noted by (Trisovic *et al.*, 2022) ensuring that research can be used, extended and verified post publishing.

6.3.3 Analytical Black Boxes

This study posits that a black box approach using a model such as a Convolutional Neural Network is the path forward for machine learning maintainability detection. These models cannot produce human interpretable rules but they can with a high accuracy predict classes for analysed data. A blanket high or low risk assessment would be an inefficient use of time and resources for hand writing code; a generated machine learning model could quickly rewrite a file based on a high risk flag without the same time commitment it would take a human.

This approach would ensure that the inefficient code patterns produced by AI noted by (Harding, 2025) are caught before even being presented to the user by running the maintainability analysis alongside the code generation tooling. A focus on recall is required to ensure all bad code gets flagged, even if ‘ghost echoes’ still appear a generative AI agent

can rewrite the code quickly and efficiently to hit the criteria set out by the black box method. This is a proposed approach for raising the code quality of research software in a sustainable effortless way.

7 Conclusion

This study sought to address the growing challenge of structural decay in research software by developing an opinionated framework of "Architectural Guardrails". By leveraging machine learning to derive these rules, this study established an empirical, domain specific truth that departs from traditional, domain agnostic industry standards.

Research software is and always has been at the forefront of innovation, the scientific community needs software to progress. Yet, it currently requires domain experts in fields such as medicine, physics and psychology to develop software engineering skills on par with software engineers to develop clean, maintainable and verifiable code. The long term solution lies in sustaining the momentum of the Research Software Engineering (RSE) movement, recognizing these experts as critical stakeholders in the research lifecycle, from funding to publication. Strides have been made to progress this initiative but until that goal is achieved we cannot neglect research software quality, the floor of research software quality has to be raised to give confidence to results being published, to allow researchers to build upon the work of each other.

Hub Score was introduced as a method to label data for maintainability risk at scale giving the ability to automatically create large high quality domain specific maintainability datasets without leaning on predetermined expert rules or hand annotation. Giving a standardised approach to create domain specific maintainability datasets for future research easily. The findings demonstrated when compared to current state of the art tooling (CodeScene) that Hub Score has comparable results when only using temporal metrics as when expert rules are used to identify high risk files. Suggesting that maintainability detection should not only use temporal data as signals to look for structural issues but temporal data should be used as a standalone diagnostic for flagging risk within files without measurable structural issues as refactoring targets.

The machine learning models trained on the file level metrics of 200+ repositories yielded mixed results for performance and a negative result for interpretability. The overall F1 score of both the Decision Tree and Random Forest models were too low to be of any value for deriving opinionated guardrails as the false positive rate was too high to be trusted to be the single source of truth on maintainability problems within a research repository. A significant finding was the shift from 'Fair' (Hub Score) to 'Moderate' (Random Forest) Kappa score during external validation to CERN repositories. This shift proves that the model successfully learned a domain specific structural classifier matching CodeScene to a Moderate extent, confirming that measurable structural rules can be derived through machine learning when peering through an empirical, temporal lens.

Interpretable rules were a goal of the models, yet the rules derived were too complex and composed of abstract markers making them impractical for human developers. Focusing on attempting to generate interpretable rules is not a feasible approach when using a machine learning method based on structural metrics. To provide the granularity for these models

to have the chance to generate accurate predictions incomprehensible polynomial metrics are required defeating the purpose of using an interpretable model.

This disconnect between predictive accuracy and rule interpretability highlights a broader systemic challenge. If the underlying patterns of architectural decay are too high-dimensional for statistical models to translate into simple refactoring targets, it is unreasonable to expect researchers, whose primary expertise lies in their scientific domain, to manually decode and rectify these structural complexities.

As every facet of software moves closer to AI, to achieve sustainable programming standards the focus must shift from human-centric linting to AI-driven automation. Utilising analytical black box models such as CNNs to flag high risk files within the generation loop requiring the agents to resolve the problems before even presenting them to the researcher is an effortless way to raise the software floor quality without burdening researchers.

This study proposes that through the use of Hub Score annotated data alongside the current Sota structural analysis of files a model could be trained to achieve this. This paradigm shift ensures that research software remains a robust foundation for scientific discovery, verification, and extension in the decades to come.

7.1 Contributions

7.1.1 Primary Contributions

The Hub Score Methodology: This study has successfully demonstrated that historical repository data can be used to identify architectural risk without the need for hand annotation of data. By benchmarking against industry standards (CodeScene), this research proves that a temporal first approach provides a domain specific "ground truth" that is an alternative to expert rules for flagging maintainability risk in files.

Evaluation Framework for Interpretable Guardrails: This study establishes a systematic approach to evaluate whether architectural decay can be distilled into human-readable rules. By attempting to derive interpretable guardrails from structural data using a machine learning pipeline, and comparing the model's performance to both temporal and structural datasets, the viability of opinionated rules can be assessed.

Evidence of the Interpretability Utility Tradeoff: This study provides evidence of the limits of using machine learning models to derive human interpretable rules for static analysis in research software. By attempting to derive interpretable guardrails, this research proves that the use of machine learning methods to predict architectural decay generates high complexity rules too impractical for general applicability. This provides a data-backed justification towards a shift to black-box AI approaches.

7.1.2 Secondary Contributions (Technical Artifacts)

In addition to the conceptual findings, this study contributes a functional technical artifact developed to facilitate large-scale repository analysis. A specialized tool developed to

automate the extraction of composite temporal and structural metrics providing: Co change graph creation: The tool implements the Neo4j to create a co change graph of all files over the history of the repository. Using the data to then assign a Hub Score to each file within the graph, allowing quick and efficient queries on the database for further analysis. Empirical Threshold Derivation: To ensure objective labeling, the tool implements the Jenks natural breaks algorithm, This allows for the autonomous derivation of a natural breaking point within any Hub Score distribution on a per dataset basis,establishing a mathematically grounded high-vs-low risk classification. Dataset creation: The tool integrates RCA to generate datasets. It maps file-level structural metrics directly to Hub Score labels, producing standardized CSV outputs that bridge the gap between static code characteristics and historical maintenance risk. Extensibility: The tool is designed to be language-agnostic and containerized, allowing other research software engineers to generate domain specific datasets for their own ecosystems.

7.2 Future Work

Future work relates to improvements in approach and method that could be implemented to progress this project.

- Historical file analysis: The method in which this project retrieved files to analyse was only retrieving files from the HEAD of each repository, a method to retrieve and label the deleted files could lead to a richer dataset with deleted being a metric to inform Hub Score accuracy as well as increasing sample size.
- New Domain Application: This study was constrained to C++ on research repositories alone, a meta analysis across domains and languages could identify if temporal patterns are language and domain agnostic.
- Metric extension: This study used only file level metrics as input for machine learning models, a platform level view or class level metrics could yield better results in predictive analysis where applicable.
- Black box model: As discussed a black box model using non interpretable rules within an AI generation loop could yield very potent results in raising the floor on research software quality.

Appendix

7.3 Software and Data Availability

The bespoke analysis pipeline, Rust-based extraction tools, and Neo4j graph schemas developed for this research are available at the following repository:

github.com/ArtLiathain/masters-thesis

This repository includes the source code for the Hub Score calculation, the dataset preprocessing scripts, and the machine learning training configurations.

Bibliography

Ardito, L. *et al.* (2020) "rust-code-analysis: A Rust library to analyze and extract maintainability information from source codes," *SoftwareX*, 12, p. 100635. Available at: https://doi.org/10.1016/j.softx.2020.100635.

Bakota, T. *et al.* (2011) "A probabilistic software quality model," in, pp. 243–252. Available at: https://doi.org/10.1109/ICSM.2011.6080791.

Bertrand, S. *et al.* (2023) "Replication and Extension of Schnappinger's Study on Human-level Ordinal Maintainability Prediction Based on Static Code Metrics," *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*, pp. 241–246. Available at: https://doi.org/10.1145/3593434.3593488.

Bertrand, S., Favier, P.-A. and Andre, j.-m. (2022) "Building an Operable Graph Representation of a Java Program as a Basis for Automatic Software Maintainability Analysis," in, pp. 243–248. Available at: https://doi.org/10.1145/3530019.3534081.

Bluemke, I. and Borsukiewicz, P. (2023) "Experiments on software error prediction using Decision Tree and Random Forest algorithms", pp. 865–869. Available at: https://doi.org/10.15439/2023F363.

Boetticher, G., Menzies, T. and Ostrand, T. (2007) "\{PROMISE\} Repository of empirical software engineering data," in. Available at: https://www.semanticscholar.org/paper/%5C%7BPROMISE%5C%7D-Repository-of-empirical-software-data-Boetticher-Menzies/f74d664f0c3451bd4c31cf973a549a6dc00897c6 (Accessed: April 30, 2026).

Borg, M., Ezzouhri, M. and Tornhill, A. (2024) *Ghost Echoes Revealed: Benchmarking Maintainability Metrics and Machine Learning Predictions Against Human Assessments*. arXiv. Available at: https://doi.org/10.48550/arXiv.2408.10754.

Breiman, L. (2001) "Random Forests," *Machine Learning*, 45(1), pp. 5–32. Available at: https://doi.org/10.1023/A:1010933404324.

Cai, Y. *et al.* (2025) "Understanding Architectural Complexity, Maintenance Burden, and Developer Sentiment -A Large-Scale Study," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pp. 2176–2187. Available at: https://doi.org/10.1109/ICSE55347.2025.00168.

Chawla, N.V. *et al.* (2002) "SMOTE: Synthetic Minority Over-sampling Technique," *Journal Of Artificial Intelligence Research* [Preprint]. Available at: https://doi.org/10.1613/jair.953.

Cohen, J. (1960) "A Coefficient of Agreement for Nominal Scales," *Educational and Psychological Measurement*, 20(1), pp. 37–46. Available at: https://doi.org/10.1177/001316446002000104.

Coleman, D. *et al.* (1994) "Using Metrics to Evaluate Software System Maintainability," *Computer*, 27, pp. 44–49. Available at: https://doi.org/10.1109/2.303623.

Faragó, C., Hegedűs, P. and Ferenc, R. (2015) "Cumulative code churn: Impact on maintainability," in *2015 IEEE 15th International Working Conference on Source Code Analysis and Manipulation (SCAM)*, pp. 141–150. Available at: https://doi.org/10.1109/SCAM.2015.7335410.

Goble, C. (2014) "Better Software, Better Research," *IEEE Internet Computing*, 18(5), pp. 4–8. Available at: https://doi.org/10.1109/MIC.2014.88.

Gosain, A. and Sharma, G. (2015) "Static Analysis: A Survey of Techniques and Tools," in D. Mandal et al. (eds.) *Intelligent Computing and Applications*. New Delhi: Springer India, pp. 581–591. Available at: https://doi.org/10.1007/978-81-322-2268-2_59.

Haddouchi, M. and Berrado, A. (2025) "Forest-ORE: Mining an optimal rule ensemble to interpret random forest models," *Engineering Applications of Artificial Intelligence*, 143, p. 109997. Available at: https://doi.org/10.1016/j.engappai.2024.109997.

Hannay, J.E. *et al.* (2009) "How do scientists develop and use scientific software?," in *2009 ICSE Workshop on Software Engineering for Computational Science and Engineering*, pp. 1–8. Available at: https://doi.org/10.1109/SECSE.2009.5069155.

Harding, W. (2025) *AI Copilot Code Quality: 2025 Data Suggests 4x Growth in Code Clones - GitClear*. Available at: https://www.gitclear.com/ai_assistant_code_quality_2025_research (Accessed: April 28, 2026).

Home | CERN (no date). Available at: https://home.cern/ (Accessed: April 30, 2026).

Howison, J. and Herbsleb, J.D. (2014) "The sustainability of scientific software:," *2014* [Preprint].

ISO 25010 (no date). Available at: https://iso25000.com/index.php/en/iso-25000-standards/iso-25010 (Accessed: February 2, 2026).

Jenks, G.F. (1967) "The data model concept in statistical mapping," *International Yearbook of Cartography*, 7, pp. 186–190. Available at: https://cir.nii.ac.jp/crid/1573668925394541312.

Jiménez, R.C. *et al.* (2017) *Four simple recommendations to encourage best practices in research software*. F1000Research. Available at: https://doi.org/10.12688/f1000research.11407.1.

Johnson, B. *et al.* (2013) "Why don't software developers use static analysis tools to find bugs?," in *2013 35th International Conference on Software Engineering (ICSE)*, pp. 672–681. Available at: https://doi.org/10.1109/ICSE.2013.6606613.

Johnson, S. and Hill, M. (1978) “Lint, a C Program Checker,” in. Available at: https://www.semanticscholar.org/paper/Lint%2C-a-C-Program-Checker-Johnson-Hill/74617cffa3c6438d04aa99bef1cca415de47d0d3 (Accessed: February 8, 2026).

Journal of Open Source Software (no date). Available at: https://joss.theoj.org/ (Accessed: April 2, 2026).

Khoshgoftaar, T.M. and Allen, E.B. (2001) “Controlling Overfitting in Classification-Tree Models of Software Quality,” *Empirical Software Engineering*, 6(1), pp. 59–79. Available at: https://doi.org/10.1023/A:1009803004576.

Lal, G.R., Chen, X. and Mithal, V. (2024) *TE2Rules: Explaining Tree Ensembles using Rules*. Available at: https://arxiv.org/abs/2206.14359.

Landis, J.R. and Koch, G.G. (1977) “The Measurement of Observer Agreement for Categorical Data,” *Biometrics*, 33(1), pp. 159–174. Available at: http://www.jstor.org/stable/2529310 (Accessed: May 1, 2026).

Marwick, B. (2017) “Computational Reproducibility in Archaeological Research: Basic Principles and a Case Study of Their Implementation,” *Journal of Archaeological Method and Theory*, 24(2), pp. 424–450. Available at: https://doi.org/10.1007/s10816-015-9272-9.

Meyer, B. (2009) *Object-oriented software construction*. 2. ed., 16. print. Upper Saddle River, NJ: Prentice Hall PTR.

Oman, P. and Hagemeister, J. (1992) “Metrics for assessing a software system's maintainability,” in *Proceedings Conference on Software Maintenance 1992*, pp. 337–344. Available at: https://doi.org/10.1109/ICSM.1992.242525.

Parnas, D.L. (1972) “On the criteria to be used in decomposing systems into modules,” *Commun. ACM*, 15(12), pp. 1053–1058. Available at: https://doi.org/10.1145/361598.361623.

Pedregosa, F. *et al.* (2011) “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, 12, pp. 2825–2830.

Quinlan, J.R. (1993) *C4.5: programs for machine learning*. San Mateo, Calif: Morgan Kaufmann Publishers (The Morgan Kaufmann series in machine learning).

Schnappinger, M., Fietzke, A. and Pretschner, A. (2020) “Defining a Software Maintainability Dataset: Collecting, Aggregating and Analysing Expert Evaluations of Software Maintainability,” in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pp. 278–289. Available at: https://doi.org/10.1109/ICSME46990.2020.00035.

Schnappinger, M. *et al.* (2019) “Learning a classifier for prediction of maintainability based on static analysis tools,” in *Proceedings of the 27th International Conference on Program Comprehension*. Montreal, Quebec, Canada: IEEE Press (ICPC '19), pp. 243–248. Available at: https://doi.org/10.1109/ICPC.2019.00043.

team, T.p.d. (2020) *pandas-dev/pandas: Pandas*. Zenodo. Available at: https://doi.org/10.5281/zenodo.3509134.

Technology, I. (2015) "Neo4j, the World's Leading Graph Database," *Neo4j Graph Database* [Preprint].

Tornhill, A. (2024) *Your code as a crime scene: use forensic techniques to arrest defects, bottlenecks, and bad design in your programs*. Second edition. Edited by K. Talbot. Dallas, Texas: The Pragmatic Bookshelf (The Pragmatic programmers).

Trisovic, A. *et al.* (2022) "A large-scale study on research code quality and execution," *Scientific Data*, 9, p. 60. Available at: https://doi.org/10.1038/s41597-022-01143-6.

Weyuker, E.J. (2011) "Empirical Software Engineering Research - The Good, The Bad, The Ugly," in *2011 International Symposium on Empirical Software Engineering and Measurement*, pp. 1–9. Available at: https://doi.org/10.1109/ESEM.2011.66.

Wilson, G. (2006) "Software Carpentry: Getting Scientists to Write Better Code by Making Them More Productive," *Computing in Science & Engineering*, 8(6), pp. 66–69. Available at: https://doi.org/10.1109/MCSE.2006.122.

Wilson, G. *et al.* (2017) "Good enough practices in scientific computing," *PLOS Computational Biology*, 13(6), pp. 1–20. Available at: https://doi.org/10.1371/journal.pcbi.1005510.

Xiao, L. *et al.* (2016) "Identifying and quantifying architectural debt," in *Proceedings of the 38th International Conference on Software Engineering*. Austin Texas: ACM, pp. 488–498. Available at: https://doi.org/10.1145/2884781.2884822.